

DynNet: A Lightweight Distributed Framework for Topology Obfuscation in Dynamic Networks

Xuanbo Huang[✉], *Member, IEEE*, Kaiping Xue[✉], *Senior Member, IEEE*, Lutong Chen[✉], *Member, IEEE*,
Zixu Huang, Jian Li[✉], *Senior Member, IEEE*, and Hyundong Shin[✉], *Fellow, IEEE*

I. INTRODUCTION

Abstract—Network Topology Obfuscation (NTO) has emerged as a promising scheme to conceal the physical layout of networks, thereby preventing adversaries from targeting critical nodes or links. By constructing deceptive virtual topologies, NTO schemes can obscure essential components and deploy honeypots to mislead and detect malicious behavior. However, existing NTO methods suffer from two major limitations. First, their high computational overhead makes them impractical for dynamic environments such as vehicular networks, where rapid topology adaptation is essential. Second, most current solutions depend on centralized controllers, typically within Software-defined Networking (SDN) architectures, which limits their applicability in decentralized or ad hoc settings. To address these challenges, we propose DynNet, a distributed and probabilistic NTO framework that adapts to dynamic topologies. Each node in DynNet independently handles probing packets (e.g., packets with low time-to-live (TTL) values) by probabilistically choosing to respond truthfully, return deceptive data, or remain silent. This probabilistic framework allows nodes to rapidly update their behavior in response to topology changes with minimal overhead. To ensure consistency, we propose a bucket-based probabilistic mapping to ensure that packets from the same flow consistently trigger the same response. Our evaluation shows that DynNet maintains effective topology concealment in highly dynamic scenarios while significantly reducing computational and communication overhead. The results demonstrate its suitability for deployment in distributed, rapidly changing network environments.

Index Terms—Distributed algorithms, dynamic networks, link-flooding attacks, moving target defense, network security, network topology obfuscation (NTO).

Received 8 October 2025; revised 10 February 2026; accepted 4 March 2026. Date of publication 12 March 2026; date of current version 31 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62201540, Grant 62302472, and Grant 62372425, and in part by the Youth Innovation Promotion Association of the Chinese Academy of Sciences under Grant Y202093. Recommended for acceptance by Dr. Tao Xiang. (*Corresponding authors: Kaiping Xue; Hyundong Shin.*)

Xuanbo Huang, Zixu Huang, and Jian Li are with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230027, China (e-mail: hxb777@mail.ustc.edu.cn; huangzixu@ustc.edu.cn; lijian9@ustc.edu.cn).

Kaiping Xue is with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230027, China, and also with the Department of Electronic Engineering, Kyung Hee University, Gyeonggi-do 17104, Republic of Korea (e-mail: kpxue@ustc.edu.cn).

Lutong Chen is with the Network and Information Center, University of Science and Technology of China, Hefei 230026, China (e-mail: lutong98@ustc.edu.cn).

Hyundong Shin is with the Department of Electronics and Information Convergence Engineering, Kyung Hee University, Gyeonggi-do 17104, Republic of Korea (e-mail: hshin@khu.ac.kr).

Digital Object Identifier 10.1109/TNSE.2026.3673521

NETWORK Topology Obfuscation (NTO) is an emerging defense strategy that constructs deceptive virtual topologies to conceal the true structure of a network [1], [2], [3]. By hiding critical nodes and links, NTO significantly increases the difficulty for adversaries to perform effective reconnaissance and launch targeted attacks. This concept has gained traction in both industry and government sectors [4], with adoption by critical infrastructure organizations such as the US NCCIC and the U.K. Ministry of Defence [5]. Gartner also forecasts rapid growth in the NTO market, underscoring its increasing strategic importance [6].

To illustrate the value of NTO, consider the typical steps an adversary takes during an attack [7], [8]. The first phase involves reconnaissance, scanning the target network to identify key assets such as nodes, links, and exposed services. This can be done using widely available tools like `traceroute` [9] or `Nmap` [10]. Once high-value targets are identified, the adversary proceeds to launch the attack, at which point traditional defenses such as intrusion detection systems (IDS) attempt to detect and mitigate malicious traffic. In contrast, NTO shifts the defensive posture earlier in the attack chain by disrupting reconnaissance itself. When adversaries probe the network, they receive deceptive or incomplete information, which hinders their ability to identify and reach actual critical infrastructure. This proactive deception prevents attacks before they begin, reducing both exposure and risk.

NTO schemes are typically designed with two primary objectives: *security* and *usability* [1], [2], [3]. The security objective refers to the ability of the virtual topology to conceal critical nodes and links from adversarial reconnaissance. In contrast, usability ensures that legitimate users can still make correct routing decisions within the obfuscated topology and that network operators can accurately diagnose link failures. These two goals are often conflicting, making it challenging to design NTO schemes that effectively balance both.

To address this, a range of NTO techniques have been proposed [1], [3], [11], [12], [13], [14], [15], which we categorize into three main classes. The first class prioritizes security at the expense of usability. For example, schemes like Linkbait [11], Bottleneck [13], and NetObfu [14] focus on misleading adversaries but disregard the routing and diagnostic needs of legitimate users. As a result, while effective in deception, these methods are often impractical for real-world deployment. The second class employs flow classification to

distinguish between benign and adversarial probing traffic, such as ProTo [12]. These approaches activate obfuscation only for suspicious flows, preserving usability for benign traffic. However, their effectiveness depends on detection model accuracy and typically requires centralized middleboxes for real-time traffic analysis, which limits their scalability and adaptability. The third class integrates both security and usability directly into the obfuscation algorithm, such as Nethide [1], EqualNet [2], and SpiderNet [3]. While these approaches show promise, they often involve complex and computationally intensive algorithms that struggle to adapt in highly dynamic environments. In this paper, we present **DynNet**, a distributed and probabilistic NTO scheme designed for dynamic network environments. To the best of our knowledge, DynNet is the first to simultaneously address the three challenges outlined earlier. It incorporates two key innovations: 1) *real-time, per-node obfuscation*, and 2) *bucket-based probabilistic mapping* of obfuscation behaviors. Unlike prior schemes [1], [2], [13], which rely on predetermined, computationally expensive combinations of actions generated offline, DynNet enables lightweight, adaptive obfuscation without requiring system reconfiguration. Each node independently processes probing packets (e.g., those with low time-to-live (TTL) values) by probabilistically selecting one of three actions: respond truthfully, reply with deceptive information, or remain silent.

This probabilistic framework allows nodes to rapidly update their behavior in response to topology changes with minimal overhead. For example, when the network evolves, each node re-computes its action probabilities based on updated local attributes and limited broadcast information, ensuring the obfuscation scheme remains effective under the new network state. Furthermore, DynNet guarantees consistency by using a hash-based bucket mapping mechanism, which deterministically assigns probing flows to actions so that repeated probes from the same flow receive identical responses.

We evaluate DynNet through simulation and compare its performance against two representative NTO schemes: NetHide [1] and EqualNet [2] under dynamic topology scenarios. Results demonstrate that DynNet achieves superior performance in both usability and security objectives, while significantly reducing computational overhead.

To summarize, this paper makes the following key contributions:

- 1) We propose **DynNet**, a distributed and probabilistic NTO scheme tailored for dynamic network environments, eliminating reliance on centralized control.
- 2) We introduce two novel techniques: a *node-level independent obfuscation framework* that jointly addresses security and usability, and a *bucket-based probabilistic mapping* mechanism that ensures flow-level consistency.
- 3) We conduct extensive simulations to benchmark DynNet against state-of-the-art NTO schemes and validate its effectiveness and efficiency under dynamic topologies.

The remainder of this paper is organized as follows. Section II provides background and surveys related work. Section III introduces the network model, threat assumptions, and system notations. Section IV details the design of DynNet. Evaluation results are presented in Section V. We discuss the

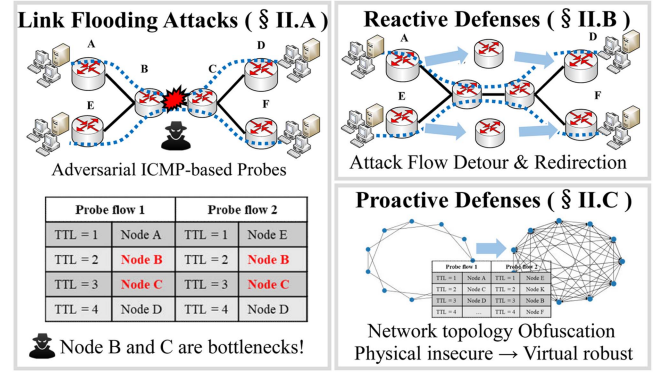


Fig. 1. Organization of Section II. We begin by reviewing link-flooding attacks in Section II-B, followed by reactive defense mechanisms in Section II-C, categorized into attack detection, intra-AS mitigation, and inter-AS mitigation. Finally, we present proactive defense strategies based on network topology obfuscation in Section II-D, highlighting state-of-the-art techniques and their differences from our proposed approach.

future direction in Section VI. Finally, Section VII concludes the paper and discusses limitations and future directions.

II. BACKGROUND AND RELATED WORK

This section provides background and related work. As illustrated in Fig. 1, the discussion is organized into three parts. First, we review the class of attacks considered in this paper. Second, we examine reactive defense mechanisms, including techniques for attack detection, intra-AS mitigation, and inter-AS mitigation. Finally, we explore proactive defenses based on network topology obfuscation, summarize state-of-the-art solutions, and clarify how our approach differs from these existing methods.

A. Background

The first link-flooding attacks (LFAs) were observed in 2013. Cloudflare reported a large-scale Distributed Denial-of-Service (DDoS) attack targeting four critical Internet Exchange Points (IXPs) in Europe and Asia [16]. The attack exhibited two notable characteristics that made it particularly effective. First, the attackers coordinated large volumes of network traffic to exhaust the bandwidth and forwarding capacity of the targeted nodes and associated links. Traditional endpoint defenses, such as firewalls and IDS, were unable to protect these core network infrastructures. Second, the targets were strategically chosen to ensure that a significant portion of global Internet traffic was affected. As a result, the attack disrupted services across multiple countries and impacted tens of providers for several weeks.

These attacks are now recognized as instances of link-flooding attacks (LFAs) [7], [8], [17]. In response, Cloudflare proposed two categories of defense strategies.

- *Reactive defense*: The straightforward response involves re-routing and load balancing to mitigate congestion at affected IXPs. This idea has since evolved into broader reactive defense strategies, including adaptive traffic engineering [18], [19], [20] and inter-AS collaboration [21], [22].
- *Proactive defense*: Cloudflare also emphasized that critical network infrastructure should not be visible to the broader

Internet in the first place. This perspective laid the foundation for proactive defense mechanisms, culminating in the development of NTO techniques [1], [3], [11], [12], [13], [14], [15].

Over the past decade, both categories of defense have been actively explored in academia and industry. Reactive defenses have matured into sophisticated traffic management solutions, while proactive defenses have advanced into robust NTO schemes. In the remainder of this section, we review recent developments in these areas. Section II-B describes representative link-flooding attacks. Section II-C discusses reactive defense mechanisms. Finally, Section II-D reviews recent work on proactive topology obfuscation schemes, which this paper aims to advance.

B. Link-Flooding Attacks

LFAs are a class of DDoS attacks that target critical network infrastructure, particularly high-value links and nodes, rather than individual services or endpoints. The concept was first introduced by Studer et al. [17] through the Coremelt attack. Coremelt demonstrated that by generating randomized traffic from distributed sources, attackers can flood the “core” of a network without directly targeting any specific service. Because these flows appear benign and are spread across multiple destinations, traditional defenses such as firewalls and IDS are ineffective. Each endpoint observes only a fragment of the total malicious traffic, making it difficult to correlate and detect the attack at a global scale.

Building on this idea, Kang et al. proposed the Crossfire attack [7], which enhances attack precision through active reconnaissance. In the first phase, the botnet sends `traceroute`-like probes to infer the underlying network topology. By measuring the *flow density*, i.e., the number of distinct flows traversing each node, the attacker identifies infrastructure components that handle a disproportionately high volume of traffic. These high-density nodes are then selected as attack targets. In a proof-of-concept evaluation, the authors showed that targeting the top 10 high-density nodes in New York could degrade over 50% of the city’s Internet traffic.

As defenses against LFAs have advanced, more sophisticated attack variants have also emerged. Tran et al. [23] proposed the concept of *rolling LFAs*, where the attacker dynamically shifts target links to exploit the high overhead associated with reactive defense mechanisms. This strategy highlights a fundamental asymmetry: while adversaries can alter their targets within seconds, defenders often require hundreds of seconds to validate and deploy updated routing configurations. More recently, Huang et al. [8] introduced the CrossPoint attack, which combines active path probing with congestion-based inference to identify bottleneck links and nodes—even in the presence of NTO defenses. By leveraging network congestion signals and recursive probing techniques, CrossPoint can effectively bypass topology obfuscation and localize critical infrastructure for targeted disruption.

In addition to traditional probing-based adversaries, several studies consider alternative adversary models. Rather than relying solely on tools such as ICMP-based probes to extract routing paths directly, attackers can employ network tomography to

infer logical topology structures or link states from end-to-end measurements [24], [25]. Sartzetakis et al. [26], [27] formulated ML-based tomography under partial topology knowledge and dynamic routing. Rkhami et al. [28] combined ML and tomography to monitor 5G network slices. Du et al. [29] employed deep spatio-temporal models to identify congested paths from end-to-end time series, and Ikeuchi et al. [30] analyzed adaptive tomography in probabilistic environments from an information-theoretic perspective. Beyond tomography, Xu et al. [31] proposed a GAN-based network reconstruction framework that recovers underlying topology from limited observations, further demonstrating that deep generative models can reconstruct hidden links.

C. Reactive Defenses

To illustrate reactive defenses, consider an LFA scenario in which attackers have identified several high-value targets across multiple autonomous systems (ASes). In response, reactive defense systems must rapidly detect the attack, localize the affected infrastructure, and coordinate rerouting at both local and global levels. Specifically, reactive defense mechanisms typically involve a three-phase process: First, the LFA detection process identifies the affected nodes or links. Second, the intra-AS mitigation mechanisms identify malicious flows and reroute traffic around congested components within a single AS. Third, inter-AS mitigation collaborates with upstream ASes to divert traffic onto alternative paths and relieve congestion.

We now review advances in each of these components, beginning with LFA detection.

LFA Detection: First, several theoretical models have been developed to understand the limits and opportunities of LFA detection. Liaskos et al. [32] demonstrated that detection performance is strongly influenced by the network’s structural properties and provided a formal analysis of how topology affects observability. Ma et al. [33] formulated the detection process as a Stackelberg security game, in which defenders adopt randomized patrolling strategies to maximize detection probability under limited resources.

Second, some approaches aim to detect the attacker’s intent during the reconnaissance phase. For instance, Li et al. [34] proposed a temporal graph learning framework to predict the evolution of probing behaviors and localize potential attack targets in advance. Huang et al. [20] designed a packet-level monitoring strategy that analyzes adversarial probing patterns to infer points of interest. These techniques reduce the defender’s monitoring overhead and allow for proactive mitigation planning.

Finally, practical detection schemes have been implemented under both traditional and Software-Defined Networking (SDN)-based architectures [35]. In traditional network environments implementation, Xue et al. [36] proposed LinkScope, which deploys multiple probing hosts to monitor delay across critical links. These hosts initiate end-to-end connections whose forwarding paths intersect at the target links. When a link becomes congested due to an LFA, multiple probing hosts observe a simultaneous increase in latency, allowing for accurate detection. With the advent of Software-Defined Networking (SDN),

more flexible and fine-grained monitoring has become possible. Wang et al. [18] presented a method that combines monitoring of reconnaissance traffic and per-link performance metrics, leveraging an SDN controller to detect anomalies and localize affected links. Ravi et al. [37] proposed a statistical congestion detection algorithm based on the variance of traffic observed at different vantage points. Their method identifies specific links experiencing abnormal congestion by evaluating flow volume imbalances across the network. Zheng et al. [38] introduced RADAR, which collects suspicious flow statistics from network switches and correlates them with path information derived from the topology database. By analyzing these patterns, RADAR can detect Crossfire-style attacks even in complex topologies. In addition, Jiang et al. [39] proposed a blockchain-assisted framework for real-time recording and sharing of link-level information, enabling efficient traceability of active flows and identification of vulnerable links.

Intra-AS mitigation: Intra-AS mitigation strategies aim to defend against LFAs through two primary objectives: 1) rerouting traffic around congested components, and 2) identifying and filtering malicious traffic flows. To achieve this goal, more flexible and fine-grained traffic engineering mechanisms have been proposed. In particular, Software-Defined Networking (SDN) [35] and programmable data planes (e.g., P4 [40]) offer enhanced visibility and control, enabling dynamic mitigation against LFAs.

Using SDN, Kang et al. [22] proposed SPIFFY, which leverages temporary bandwidth increases to identify stealthy attack flows. The key insight is that, under congestion, benign flows react to newly available bandwidth, whereas attack flows maintain constant rates to remain undetected. This behavioral difference enables discrimination between benign and malicious traffic. Wang et al. [18] introduced LFADefender, which incorporates two key modules: traffic rerouting and malicious flow blocking. The rerouting module adjusts forwarding paths to force adversaries to reissue reconnaissance probes. Concurrently, the blocking module monitors flow behaviors and filters out those exhibiting persistent probing activity, thus gradually exposing the botnet infrastructure. Woodpecker [44] demonstrated that incremental SDN deployment can maximize available path diversity for LFA mitigation. In parallel, Rezapour et al. [45] proposed RL-Shield, a reinforcement learning framework that identifies source IPs reacting persistently to rerouting events. The system employs Dirichlet distributions and Bayesian inference to model attacker adaptation and isolate malicious entities.

Recently, programmable switches are widely used in line-rate attack defenses [46], [47]. Xing et al. [48] developed Ripple, which generates switch programs to construct a distributed and panoramic view of attack patterns. This enables a fully decentralized response capable of adapting to fast-changing threats. However, P4-based systems often suffer from storage and memory limitations when facing high flow volumes. To address these constraints, Zhou et al. [49] proposed Mew, which incorporates distributed state storage and a lossless migration mechanism to counter LFAs under massive traffic. Mew ensures that the system remains effective even when attack strategies evolve rapidly. Additionally, Nance et al. [50] explored the use

of optical networking programmability to dynamically scale network capacity and create new traffic paths in response to LFA incidents, offering an orthogonal solution to congestion-based attacks.

Inter-AS Mitigation: Inter-AS mitigation mechanisms aim to block or reroute traffic at the level of upstream ASes, enabling collaborative defense against LFAs through cross-domain coordination. These approaches typically build upon extensions to inter-domain routing protocols, such as the Border Gateway Protocol (BGP), or introduce new coordination frameworks to facilitate scalable mitigation.

Lee et al. [21] proposed CoDef, a collaborative framework that combines rerouting and rate control. When a victim AS experiences link flooding, it issues rerouting requests to all source ASes responsible for sending traffic toward the congested region. In response, legitimate sources comply by detouring their traffic around the affected infrastructure. Meanwhile, the victim AS performs compliance testing to distinguish between cooperative and uncooperative flows—malicious traffic that ignores rerouting requests can thereby be identified and filtered. To support scalable cross-AS bandwidth management, Basescu et al. [51] introduced SIBRA, a hierarchical resource reservation framework. SIBRA defines three contract types, *i.e.*, core, steady, and ephemeral, to govern bandwidth allocation between ASes. This structure allows for predictable traffic shaping and efficient mitigation of LFAs by limiting attackers' ability to saturate inter-domain links. Smith et al. [52] proposed Nyx, which treats DDoS defense as a dynamic routing problem. Nyx enables a multi-homed AS to isolate and reroute critical inbound traffic from remote ASes around congested paths, leveraging existing BGP functionalities. By proactively managing inter-AS routes, Nyx enhances the flexibility of upstream mitigation without requiring fundamental protocol changes. More recently, Lu et al. [53] proposed STOP, a service-oriented DDoS mitigation architecture. In STOP, participating ASes offer traffic filtering as a paid service, allowing customers to deploy filtering nodes based on demand selectively. The architecture supports incremental deployment and incentivizes adoption through economic mechanisms, including blockchain-backed payment verification. This model encourages wider participation in collaborative filtering while reducing reliance on global protocol upgrades.

D. Network Topology Obfuscation

NTO aims to disrupt adversarial reconnaissance by constructing a virtual topology that conceals critical nodes and links. NTO can be viewed as a specific form of cyber deception that operates at the topology layer [54]. The fundamental idea is to present an adversary with a misleading yet functionally consistent view of the network. Designing a practical NTO scheme requires achieving two primary objectives: *security* and *usability* [1], [2], [3]. The security objective focuses on the ability of the virtual topology to hide sensitive network elements and mitigate adversarial inference. Conversely, the usability objective ensures that legitimate users can make correct routing decisions and that operators can accurately diagnose link failures. These objectives are inherently conflicting, making it challenging to

TABLE I
COMPARISON OF DYNNET WITH RELATED WORKS IN TERMS OF DESIGN OBJECTIVES AND NETWORK MODEL

Method	Design Objective			Framework Architecture	Adversary Model			Network Model	Benign Flow Classification	Running Overhead
	Sec.	Acc.	Uti.		Blind	ICMP	Delay			
Linkbait [11]	●	○	○	Centralized	●	●	○	Static	Yes	$O(\mathcal{N} ^2 F)$
BottleNet [13]	●	○	○	Centralized	●	●	○	Static	No	$O(\mathcal{N} ^2)$
NetObfu [14]	●	○	○	Centralized	●	●	○	Static	No	$O(\mathcal{N} \mathcal{F})$
Tan <i>et al.</i> [41]	●	○	○	Distributed	Not discussed			Dynamic	No	$O(\mathcal{N} \mathcal{E})$
EigenObfu [15]	●	○	○	Centralized	Not discussed			Static	No	$O(\mathcal{N} ^3)$
ChameleonNet [42]	●	○	○	Centralized	○	○	●	Static	No	- [†]
Proto [43]	●	○	○	Centralized	○	○	●	Static	Yes	- [†]
SpiderNet [3]	●	●	○	Centralized	●	●	○	Static	No	$O(\mathcal{N} \mathcal{F})$
NetHide [1]	●	●	●	Centralized	●	●	○	Static	No	$O(k^{ \mathcal{N} })$
EqualNet [2]	●	●	●	Centralized	●	●	○	Static	No	$O(\mathcal{N} \mathcal{F})$
DynNet	●	●	●	Distributed	●	●	○	Dynamic	No	$O(\mathcal{F})$

[†] For Proto and ChameleonNet, runtime overhead is not directly comparable due to differences in their adversary models. For Nethide, k denotes the input parameter, representing the number of forwarding trees per node, typically set to 100. The ○ indicates not considered, whereas ○ denotes features that are orthogonal or compatible with another one.

design schemes that provide strong security guarantees without compromising operational functionality.

To address this challenge, a range of NTO techniques have been proposed [1], [11], [12], [13], [14], [15], [41], [55], [56], [57]. Table I summarizes and compares these approaches in terms of design objectives, framework architecture, adversary model, network model, benign-flow classification strategy, and runtime overhead. In the following, we categorize existing schemes into three main classes based on their design objectives.

The first class prioritizes security at the expense of usability [11], [13], [14], [15], [41], [55], [56]. Achleitner et al. [58] designed a Reconnaissance Deception System that leverages SDN to assign host-specific virtual network views. Ding et al. [11] proposed Linkbait, which selects a group of secure links as bait to make them exposed to attackers. Linkbait formulates the bait-link selection problem as a weighted set cover problem to maximize the coverage of probing flows while minimizing link costs. Kim et al. [13] proposed BottleNet, a centralized defense system built on SDN infrastructure that uses ILP-based optimization to select deployment points and virtual links so as to maximize deception while minimizing deployment cost. Liu et al. [14] introduced NetObfu, which computes a secure virtual topology and enables each node to forge responses locally without modifying real forwarding paths, thereby preserving the original routing while confusing reconnaissance traffic. Tan et al. [41] designed a topology deception scheme for Uncrewed Aerial Vehicle (UAV) networks, where SDN controllers dynamically configure virtual links and paths to steer attacks away from key drones. Liu et al. [55] proposed AntiTomo, a tomography-aware obfuscation mechanism that generates candidate routing forests and then solves a multi-objective optimization problem to select an obfuscated topology. Qin et al. [59] propose a CGAN-based cyber deception framework for industrial control systems. Zhu et al. [15] developed EigenObfu, which uses eigenvector centrality to identify critical nodes and applies eigenvalue-preserving rewiring to reshape the topology. Li et al. [56] propose a high-order camouflaged NTO

scheme that extracts a hypergraph-based backbone using statistical validation of hypergraphs to predict backbone evolution. However, these schemes largely optimize security objectives and typically treat usability constraints only implicitly or at a coarse level, which may cause benign users to face increased path stretch or diagnosis difficulty under strong obfuscation.

The second class employs flow classification to distinguish between benign and adversarial probing traffic. Hou et al. [12], [43] designed ProTO, which uses a machine learning-based classifier to identify attackers' probe packets. The topology obfuscation module then selectively perturbs the forwarding paths and timing of the identified probes, leading attackers to acquire false network topology information based on delayed or distorted measurements. Alanazi et al. [60] introduced an adaptive neural framework that combines dynamic feature selection with an attention-based LSTM to capture temporal dependencies and enhance detection performance on evolving probing patterns. These schemes improve security by focusing obfuscation on suspected probes. However, their effectiveness depends heavily on detection model accuracy and typically requires centralized middleboxes for real-time traffic analysis, which limits their scalability and adaptability in large-scale or highly dynamic environments.

The third class integrates both security and usability directly into the obfuscation algorithm, such as NetHide [1], EqualNet [2], SpiderNet [3], and OptiPathNet [57]. NetHide [1] formulates usability as an explicit optimization objective while encoding security requirements as constraints, thereby searching for virtual topologies that remain close to original routing for benign users. EqualNet [2] further balances path exposure across nodes and links to mitigate link-flooding threats, explicitly constraining flow-density variance while enforcing indistinguishability between decoy and real paths. SpiderNet [3] leverages fine-grained per-flow obfuscation and introduces honeylinks to identify link-flooding bots. OptiPathNet [57] uses a GCMOEA-based multi-objective evolutionary optimizer to jointly balance security, latency, and energy consumption. While

these approaches are closer to joint security–usability design, they often involve complex and computationally intensive algorithms or centralized coordination, and they still struggle to achieve decentralization, lightweight operation, and seamless adaptation in highly dynamic environments.

Prior NTO systems [1], [2], [3] follow a global construction framework: they compute an obfuscated topology based on a global network-wide view and then distribute the configuration to all nodes. This design is effective when topology and traffic are relatively stable and when centralized computation and orchestration are timely. In contrast, DynNet follows a distributed control framework. Each node maintains a local probabilistic control policy and updates it using local observations and lightweight coordination signals, without requiring global re-optimization. This difference yields two advantages. First, the incremental maintainability. Topology or traffic changes trigger only localized policy updates rather than a network-wide recomputation and redistribution. Second, the robustness against a single failure. Nodes can continue enforcing obfuscation even when centralized control is delayed or unavailable.

To defend against tomography-based inference attacks as mentioned in Section II-B, ProTO [12], [43] employed a machine-learning-based classifier to identify and selectively obfuscate attackers’ probe packets. AntiTomo [55] focused specifically on degrading tomography accuracy by carefully distorting path-level correlations, while Li et al. [56] addressed temporal composition inference using high-order camouflaged obfuscation. Qiu et al. [42] proposed ChameleonNet, a defense framework that obscures topological features from tomography-based adversaries by dynamically adjusting routing and measurement responses. In this paper, we focus on a more general adversary model, where attackers are assumed to have access to active probing tools such as `traceroute` for topology discovery and may further combine traditional probes with advanced ML-based inference.

III. MODEL

A. Network Model

We consider a network operated by a single administrative entity, such as an Internet Service Provider (ISP). The network topology evolves over time due to dynamic changes in connectivity. Specifically, we define topology dynamics as changes in the edge set. Links between nodes may be added or removed over time. Such edge-level variation is common in mobile and wireless environments, including UAV or vehicular networks [61]. We assume that the set of network nodes remains constant throughout the time horizon. This assumption is reasonable for many practical deployments where the infrastructure nodes (*e.g.*, routers, base stations) are relatively stable, while their interconnections vary due to mobility, fading, or handovers [62], [63]. Moreover, we assume the network topology remains connected at all times. This does not reduce generality, as disconnected components can be treated independently using the same obfuscation framework. In addition, we assume that the data plane has the following capabilities: (i) truthful replies, where the TTL is decremented and a normal

TABLE II
SUMMARY OF NOTATION USED IN THE FORMULATION AND METRICS

Notation	Description
$\mathcal{N}$	Set of nodes in the topology, $\mathcal{N} = \{n_1, n_2, \dots\}$.
$\mathcal{E}$	Set of links in the topology, $\mathcal{E} = \{e_1, e_2, \dots\}$.
$\mathcal{P}$	Physical topology, $\mathcal{P} = (\mathcal{N}, \mathcal{E})$.
$\mathcal{V}$	Virtual (obfuscated) topology, $\mathcal{V} = (\mathcal{N}^v, \mathcal{E}^v)$.
$\mathcal{F}$	Set of flows in the network, $\mathcal{F} = \{f_1, f_2, \dots\}$.
$f_{(s,d)}$	A flow from source s to destination d , where $s, d \in \mathcal{N}$.
$R(f)$	Routing path of flow f in the physical topology.
$R^v(f)$	Obfuscated routing path of flow f in the virtual topology.
$fd(\mathcal{N}_i)$	Number of flows traversing $\mathcal{N}_i$ in the physical topology $\mathcal{P}$.
$fd^v(\mathcal{N}_i)$	Number of flows traversing $\mathcal{N}_i$ in the virtual topology $\mathcal{V}$.
FDP, FD^v	Total flow density in the physical and virtual topologies.
STD_{fd}	Standard deviation of node-level flow density.
CV_{fd}	Coefficient of variation of flow density.
$\mathbb{E}[\cdot]$	The expected (average) value.
α_i	Probability that node i remains silent during obfuscation.
β_i	Probability that node i responds with a fake identity.
γ_i	Probability that node i responds truthfully in obfuscation.
θ	Input parameter controlling the obfuscation level.
$H(\cdot), L$	Hash function and its output length L .

ICMP Time Exceeded message is generated, (ii) deceptive replies, where the TTL is decremented but the ICMP header fields are rewritten to emulate another node, and (iii) silent forwarding, where the probe is forwarded without decrementing the TTL and no ICMP response is sent. Such manipulations are consistent with the programmable data-plane assumptions in prior topology obfuscation systems [1], [2], [14].

Table II summarizes the key notation used throughout this paper. We denote the physical topology as $\mathcal{P} = (\mathcal{N}, \mathcal{E})$, where $\mathcal{N}$ and $\mathcal{E}$ represent the sets of nodes and links, respectively. The obfuscated virtual topology is denoted as $\mathcal{V} = (\mathcal{N}^v, \mathcal{E}^v)$. Network probing traffic is modeled as a set of flows $\mathcal{F}$, where each flow $f_{(s,d)}$ follows a routing path $R(f)$ in the physical topology and an obfuscated path $R^v(f)$ in the virtual topology. We define $fd(\mathcal{N}_i)$ and $fd^v(\mathcal{N}_i)$ as the flow densities of node $\mathcal{N}_i$ in the physical and virtual topologies, respectively, and FDP, FD^v as their corresponding global sums. Flow dispersion is captured using the standard deviation STD_{fd} and the coefficient of variation CV_{fd} . The triplet $(\alpha_i, \beta_i, \gamma_i)$ specifies the obfuscation policy for node i , where α_i denotes the probability of remaining silent, β_i of providing a fake response, and γ_i of replying truthfully. The global parameter θ controls the overall obfuscation level. Finally, the hash-based mapping relies on a hash function $H(\cdot)$ and its output length L . These definitions provide the foundation for the formulation and evaluation presented in subsequent sections.

B. Threat Model

We assume that the adversary controls a subset of the network flow set $\mathcal{F}$, using these flows as part of a botnet infrastructure. In the worst case, the adversary may control the entire set $\mathcal{F}$. The primary objective of the adversary is to identify critical nodes and links within the network topology to launch LFAs

that degrade the communication quality or availability of the targeted area.

During the reconnaissance phase, the adversary employs path tracing tools such as `traceroute` [9], `MTR` [64], and similar utilities. These tools send probe packets with incrementally increasing Time-To-Live (TTL) values. When a packet's TTL reaches zero at an intermediate router, the router returns an ICMP Time Exceeded response to the packet source, thereby revealing its position in the path [65]. By aggregating these responses across multiple flows, the adversary can reconstruct the underlying forwarding paths. In real networks, ICMP probes may be filtered, rate-limited, or lost, and routing dynamics may cause path measurements to become stale. However, we adopt a worst-case reconnaissance assumption. We assume that probes are neither filtered nor dropped, and that the adversary can repeat probing frequently enough for the inferred paths to accurately reflect the current topology. This provides an upper bound on the attacker's knowledge and thus a conservative setting for evaluating our defense.

To identify critical infrastructure, the adversary performs a statistical analysis on the collected probing paths. Following the Crossfire attack model [7], the adversary computes the flow density of each node and link. Flow density is defined as the number of probing paths traversing that node or link. Intuitively, nodes and links with high flow density are likely to appear on many end-to-end paths and thus play a central role in overall network connectivity. In our model, the adversary can update these density estimates for each snapshot, so the set of critical nodes and links is always computed from an up-to-date view of the network.

IV. DYNNET

A. Overview

NTO aims to construct a robust virtual topology over the physical network to defend against adversarial reconnaissance on critical infrastructure such as nodes and links. Formally, given a physical topology graph $\mathcal{P}$, the objective is to generate a virtual topology graph $\mathcal{V}$ that satisfies both security and usability goals (detailed later).

Conventional NTO approaches rely on complex combinations of strategies to construct virtual topologies. Typical methods include skipping physical nodes [1], adding virtual nodes [13], creating virtual links [2], or preserving the original topology. To identify an optimal solution, traditional schemes often employ centralized optimization, leveraging global information to produce comprehensive strategies. However, this leads to significant computational overhead and a single point of failure. For instance, `NetHide` [1] has a worst-case complexity of $O(k^{|N|})$, where k is an input parameter typically equal to 100. `EqualNet` and `SpiderNet` require enumerating all nodes and flows [3]. Such costs render these solutions impractical for dynamic topologies or networks lacking high-performance centralized controllers.

To address these limitations, we propose **DynNet**, a distributed NTO scheme designed for scalability and resilience. Instead of centralized control, DynNet abstracts the decision space at each node into three possible actions: remain silent

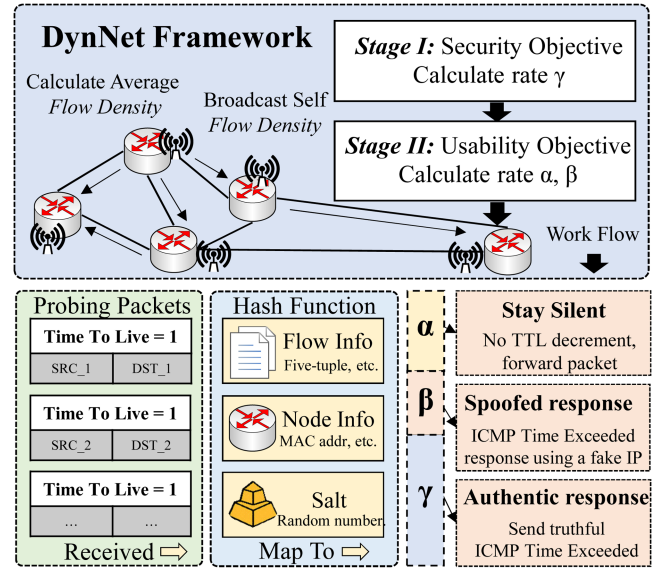


Fig. 2. Overview and workflow of DynNet. DynNet determines the probability triplet (α, β, γ) through a two-stage process: *Stage I* optimizes security by computing γ , while *Stage II* optimizes usability by deriving α and β . During operation, each node applies these probabilities using a hash-based mapping mechanism that deterministically assigns probing packets to corresponding behaviors (stay silent, spoofed response, or authentic response).

(skip), respond with falsified information (create virtual nodes), or return truthful data to preserve usability. Each node independently selects an action for incoming flows according to predefined probabilities. Thus, the core challenge is to determine the optimal probability distribution across these three actions to balance security and usability.

Fig. 2 gives the overview and the workflow of DynNet. DynNet computes these probabilities through a two-stage process. In the first stage, we formalize the security objective as a convex quadratic optimization problem and solve it using a lightweight heuristic algorithm. The algorithm outperforms existing solvers. In the second stage, we estimate node-level usability weights to guide the trade-off between silence and false responses. Intuitively, silence reduces routing options, whereas false responses can distort fault diagnosis. To mitigate usability loss, we assess each node's contribution to routing and fault detection, then apply a softmax-based aggregation to derive its action probabilities.

This distributed design provides significant computational advantages. For each node, DynNet operates with complexity $O(|F_n|)$, where $|F_n|$ denotes the number of flows traversing the node, compared to $O(|N||F|)$ for `SpiderNet` and `EqualNet`, and $O(K^{|N|})$ for `NetHide`. Moreover, DynNet requires minimal inter-node communication, involving only a lightweight broadcast of average flow density. This adaptability is achieved through localized computation and minimal state exchange: nodes only require a lightweight broadcast of updated flow density to refresh their probability triplets. When the network state changes, DynNet retrieves the new flow routing table and recalculates usability weights to adjust response probabilities. Consequently, DynNet can quickly adjust obfuscation behavior

to maintain security and usability guarantees even under frequent topology changes.

In the following subsections, we formally define the design objectives of DynNet and then detail its two-stage optimization process and implementation.

B. Design Objectives

The primary objective of DynNet is to jointly optimize *security* and *usability* within the obfuscation process. In this subsection, we formalize these objectives and describe how they are quantitatively evaluated.

Security Objective: The security of an NTO scheme is evaluated using two key metrics: (i) the ability to reduce flow density across the network, and (ii) the indistinguishability of critical nodes. We elaborate on these metrics below.

a) Flow Density Reduction factor: As described in the threat model, adversaries often infer node importance from flow density, defined as the number of probing flows traversing a node. Reducing overall flow density helps obscure high-value nodes and disperses adversarial attention. Therefore, our first design goal is to lower the flow density across all nodes to an acceptable range, which can be controlled through input parameters that determine the desired level of obfuscation. Specifically, we define flow density as follows:

Let $fd(\mathcal{N}_i)$ represent the number of flows traversing a physical node $\mathcal{N}_i$ in the real topology $\mathcal{P}$:

$$fd(\mathcal{N}_i) = \sum_{f \in \mathcal{F}} \mathbb{1}_{\mathcal{N}_i \in R^p(f)}, \quad (1)$$

where $R^v(f)$ denotes the obfuscated routing path of flow f , and $\mathbb{1}_{\mathcal{N}_i \in R}$ is an indicator function equal to 1 if $\mathcal{N}_i$ appears in path R , and 0 otherwise. The total flow density in the physical topology is then:

$$FD_p = \sum_{\mathcal{N}_i \in \mathcal{P}} fd(\mathcal{N}_i). \quad (2)$$

Similarly, let $fd^v(\mathcal{N}_i)$ denote the number of flows observed at node $\mathcal{N}_i$ in the obfuscated topology $\mathcal{V}$, with:

$$FD_v = \sum_{\mathcal{N}_i \in \mathcal{V}} fd^v(\mathcal{N}_i). \quad (3)$$

The *flow density reduction factor* is defined as:

$$\Delta_{FD} = 1 - \frac{FD_v}{FD_p}, \quad \Delta_{FD} \in [0, 1]. \quad (4)$$

This metric measures the extent to which an NTO scheme reduces the adversary's ability to infer the true topology. Importantly, Δ_{FD} is tunable through user-defined parameters, enabling a controlled trade-off between security and usability. For example, NetHide adjusts Δ_{FD} via constraint thresholds [1], while EqualNet varies its obfuscation threshold to achieve different security levels [2]. DynNet adopts a similar approach, allowing flexible configuration. Notably, a higher Δ_{FD} is not always desirable, as excessive reduction may compromise usability. Therefore, an effective scheme should enhance both security and usability simultaneously, as demonstrated in Section V.

b) Flow Density Standard Deviation: To further reduce the risk of critical node exposure, an ideal obfuscation scheme should distribute flow density as uniformly as possible across all nodes, so that each appears equally important. Thus, our second objective is to minimize the standard deviation of node-level flow densities:

$$\text{STD}_{fd} = \sqrt{\frac{1}{|\mathcal{N}|} \sum_{\mathcal{N}_i \in \mathcal{N}} (fd(\mathcal{N}_i) - \mathbb{E}[fd(\mathcal{N})])^2}, \quad (5)$$

where $\mathbb{E}[fd(\mathcal{N})] = \frac{1}{n} \sum_{i=1}^n fd(\mathcal{N}_i)$ is the average flow density. A lower STD_{fd} indicates a more balanced flow distribution, making it harder for adversaries to identify strategically important nodes and thereby strengthening obfuscation. Since the standard deviation is influenced by the mean, we adopt the coefficient of variation as a normalized metric:

$$CV_{fd} = \frac{\text{STD}_{fd}}{\mathbb{E}[fd(\mathcal{N})]}. \quad (6)$$

Usability Objective: Usability reflects the ability of benign users and network operators to reason about the network state using the virtual topology. While obfuscation enhances security, it must not render the network unusable. Consider an extreme obfuscation policy where all nodes remain silent and refuse to respond to probing traffic. Such a design would guarantee strong security, as adversaries could learn nothing about the network. However, this approach severely impairs legitimate operations: network operators would be unable to perform essential tasks such as diagnosing failures or making informed routing decisions. Therefore, an effective NTO scheme must balance obfuscation with the need to maintain sufficient visibility for operational purposes.

To quantify usability, prior work [1] introduces two key metrics: Accuracy and Utility. These metrics are widely used and capture how closely the virtual topology approximates the physical topology in terms of logical and physical consistency, respectively.

c) Accuracy: Accuracy [1] measures the logical similarity between routing paths reported under the original topology and those observed through the obfuscated topology. A virtual topology with high Accuracy allows operators to diagnose routing issues such as suboptimal paths. Conversely, low Accuracy may cause traceroute results to show unrealistic paths, such as traffic seemingly jumping between geographically distant nodes, leading to misdiagnosis. Formally, the Accuracy of a flow f is defined as:

$$\text{acc}(f) = 1 - \frac{\text{Lev}(R(f), R^v(f))}{|R(f)| + |R^v(f)|}, \quad (7)$$

where $\text{Lev}(\cdot, \cdot)$ is the Levenshtein distance [66], and $|R(f)|$ is the length of flow f 's path. The Levenshtein distance computes the minimum number of insertions, deletions, or substitutions required to transform one sequence into another. Following NetHide [1], we model each path as an ordered sequence of nodes and compute the Levenshtein distance over these node symbols, not over the character strings of IP or MAC addresses. In our implementation, each switch is mapped to a unique integer ID by the controller, and paths are compared as sequences of

these IDs. As long as this mapping is consistent, the distance is independent of whether the underlying labels are represented as IPs, MACs, or internal identifiers. In this context, it reflects how much the obfuscation distorts the original route: larger distances imply greater deviation and lower usability. The overall Accuracy of the obfuscated topology is the average across all flows:

$$ACC(\mathcal{P}, \mathcal{V}) = \mathbb{E}[acc(f)], \quad f \in \mathcal{F}. \quad (8)$$

d) Utility: Utility measures the physical similarity between the paths observed in the physical topology and those in the virtual topology. Intuitively, this metric captures how accurately the virtual topology reflects real-world conditions such as link failures or congestion. If a physical link fails, a virtual topology with high utility should enable operators to detect and reason about this event. Formally, the utility for a flow from s to d is:

$$u(f_{(s,d)}) = \frac{1}{|R^v(f_{(s,d)})|} \sum_{n \in R^v(f_{(s,d)})} u_n, \quad (9)$$

where:

$$u_n = \frac{1}{2} \left(\frac{|C|}{|R(f_{(s,n)})|} + \frac{|C|}{|R^v(f_{(s,d)})[0:n]|} \right), \quad (10)$$

and:

$$C = R(f_{(s,n)}) \cap R^v(f_{(s,d)})[0:n]. \quad (11)$$

Here, $R(f_{(s,n)})$ denotes the set of links on the physical path from s to n , $R^v(f_{(s,d)})[0:n]$ represents the prefix of the virtual path from s to d up to node n , and C is the set of common links. The overall network utility is then defined as:

$$UTI(\mathcal{P}, \mathcal{V}) = \mathbb{E}[u(f_{(s,d)})], \quad f_{(s,d)} \in \mathcal{F}. \quad (12)$$

C. Problem Statement

The key problem of DynNet is as follows: How can each node n determine its optimal response strategy to balancing security and usability based on local and global network information?

Specifically, each node receives the following inputs: (1) its routing table, (2) the set of active flows currently traversing node n , (3) the average flow density of the entire network, and (4) an obfuscation control parameter θ (discussed later), which specifies the desired level of obfuscation.

Using these inputs, node n must compute a probability distribution over three possible actions: (1) the probability of returning truthful information $\gamma_n \in [0, 1]$, (2) the probability of remaining silent $\alpha_n \in [0, 1]$, and (3) the probability of returning falsified information $\beta_n \in [0, 1]$. Subject to the constraint:

$$\alpha_n + \beta_n + \gamma_n = 1. \quad (13)$$

The objective is to select these probabilities in a way that jointly optimizes security and usability. The following sections describe how DynNet achieves this through a two-stage optimization process.

D. Stage I: Truthful Exposure Balancing

DynNet decomposes the obfuscation problem into two sub-problems. In Stage I, the goal is to compute γ_n , the probability

that node n returns truthful information when queried. Since the primary security objective of NTO is to reduce the overall flow density observed in the network, we first define a global parameter that can directly control the proportion of visible flow in the obfuscated topology. We introduce the global control parameter θ , defined as the ratio of total visible flow in the virtual topology to that in the physical topology:

$$\theta = \frac{\sum_{n \in \mathcal{N}} fd^v(\mathcal{N})}{\sum_{n \in \mathcal{N}} fd(\mathcal{N})} = \frac{\mathbb{E}(fd^v(\mathcal{N}))}{\mathbb{E}(fd(\mathcal{N}))}. \quad (14)$$

Here, $fd(\mathcal{N})$ represents the flow density of node n in the physical topology, and $fd^v(\mathcal{N})$ denotes its exposed flow density in the virtual topology. Intuitively, θ captures the proportion of flow information retained after obfuscation. It serves as a global tuning parameter that determines the trade-off between security and usability:

- When $\theta \rightarrow 0$, the system enforces strong obfuscation by heavily reducing visibility, maximizing security at the cost of usability.
- When θ approaches its upper bound, the virtual topology closely mirrors the physical topology, maximizing usability but providing minimal obfuscation.

To ensure flexibility, θ is bounded as:

$$\theta \in \left(0, \frac{\max_{n \in \mathcal{N}} fd(\mathcal{N})}{\mathbb{E}(fd)} \right], \quad (15)$$

where $\max_{n \in \mathcal{N}} fd(\mathcal{N})$ is the maximum node flow density in the physical topology, and $\mathbb{E}(fd)$ is the average flow density. This upper bound can exceed 1 to guarantee feasibility in extreme cases. Physically, when θ equals the upper bound, all nodes respond truthfully, and the virtual topology is identical to the physical topology (zero obfuscation). Conversely, when θ approaches 0, the topology becomes highly concealed, maximizing security but severely degrading usability.

While θ defines a global visibility constraint, the challenge lies in translating this global goal into per-node decisions. Our second security objective, as discussed earlier, is indistinguishability: after obfuscation, the flow density distribution should be as uniform as possible, minimizing an adversary's ability to infer critical nodes. Simply applying a uniform probability γ across all nodes would still result in high-load nodes being overrepresented, making them easily identifiable. To address this, we introduce node-specific truthful response probabilities γ_n and formalize the following optimization problem:

$$\min \text{STD}_{fd} \quad \text{s.t. } \gamma_n \in (0, 1), \quad \forall n \in \mathcal{N}. \quad (16)$$

Rewriting the objective as the squared deviation, we obtain:

$$\begin{aligned} & \text{minimize} \quad \sum_{n \in \mathcal{N}} (\gamma_n \cdot fd(\mathcal{N}) - \mathbb{E}(\gamma_n \cdot fd(\mathcal{N})))^2, \\ & \text{subject to} \quad \gamma_n \in (0, 1), \quad \forall n \in \mathcal{N}. \\ & \quad \quad \quad fd^v(\mathcal{N}) = \gamma_n fd(\mathcal{N}), \quad \forall n \in \mathcal{N}. \\ & \quad \quad \quad \sum_{n \in \mathcal{N}} fd^v(\mathcal{N}) = \theta \sum_{n \in \mathcal{N}} fd(\mathcal{N}). \end{aligned} \quad (17)$$

This convex formulation seeks to minimize variance in exposed flow density while respecting the global constraint. The optimal solution assigns a smaller γ_n to high-load nodes and a larger γ_n to low-load nodes, balancing visibility across the network.

Practical Approximation: Since DynNet delegates the decision of γ_n to individual nodes, solving the convex optimization problem in a centralized manner would be inefficient and incompatible with the distributed design. To address this, DynNet employs a lightweight heuristic that allows each node to compute its own truthful exposure probability while strictly satisfying the global constraint imposed by θ . The heuristic is defined as:

$$\gamma_n = \min \left(1, \frac{\theta \cdot \mathbb{E}(fd)}{fd(\mathcal{N})} \right), \quad (18)$$

where $\mathbb{E}(fd)$ is the average flow density of the network, and $fd(\mathcal{N})$ is the flow density of node n . This design ensures that the global exposure ratio converges to θ . In addition, high-load nodes are assigned smaller γ_n values to reduce their dominance in probing paths. In particular, very low-load nodes can obtain $\gamma_n = 1$ and always respond truthfully. This is acceptable under our threat model, because such nodes carry little traffic and are unlikely to be profitable targets for LFAs. Their contribution to the overall flow-density distribution is small, so exposing them does not significantly affect the security metric, but it helps maintain accurate path information for benign diagnostic tasks.

Example: Consider a network with two nodes: $fd(1) = 100$, $fd(2) = 80$, and $\mathbb{E}(fd) = 90$. For $\theta = 0.5$, we compute $\gamma_1 = \min(1, \frac{0.5 \cdot 90}{100}) = 0.45$, and $\gamma_2 = \min(1, \frac{0.5 \cdot 90}{80}) = 0.5625$. Node 1, being more heavily loaded, truthfully responds for $45\% \cdot 100 = 45$ flows, while Node 2 responds for $56.25\% \cdot 80 = 45$ flows. This allocation perfectly equalizes exposure across nodes, driving the standard deviation of flow density to zero and thus achieving the indistinguishability objective.

The heuristic is computationally lightweight and can be executed locally at each node. Each node only needs to maintain its own $fd(\mathcal{N})$ and periodically update $\mathbb{E}(fd)$ using network-wide broadcasts of flow density values. After receiving these updates, the node recalculates its γ_n without any centralized coordination. Moreover, this practical approximation significantly outperforms modern solvers such as Gurobi [67] and CVX [68], [69], as demonstrated in Section V.

Node update: DynNet is analyzed under a fixed node set with time-varying edges. If nodes are added or removed, DynNet recomputes the global average flow density $\mathbb{E}(fd)$ and updates γ_n at all nodes accordingly. Thanks to its distributed design and lightweight computation, DynNet can perform these updates quickly in practice.

E. Stage II: Obfuscation Policy Decomposition

In Stage II, our objective is to optimize the **usability metric**, which consists of two components: Utility and Accuracy. Utility reflects event observability (e.g., the ability to detect link failures), while Accuracy measures the structural similarity between physical and virtual paths, ensuring routing fidelity. Given the known truthful response probability γ_n for each node, we aim

to compute the silence probability α_n and the false-response probability β_n , subject to the constraint:

$$\alpha_n + \beta_n = 1 - \gamma_n. \quad (19)$$

Before introducing the computation of these parameters, we analyze how silence and false-response behaviors affect the usability metrics.

Impact of Silence and False Responses: This trade-off was first examined by NetHide [1]. The Utility metric (9) captures whether failures in the physical topology remain observable after obfuscation. If the virtual path retains structural similarity to the physical path, failure signals remain detectable. In contrast, Accuracy (7) measures path similarity and is crucial for routing decisions because it determines whether the obfuscated path adheres to performance constraints such as avoiding congestion and maintaining QoS.

However, silence and false responses affect these two metrics in fundamentally different ways:

1) *Silence preserves Utility but harms Accuracy:* Silence does not introduce incorrect links, so the common link set between physical and virtual paths remains mostly intact, preserving Utility. However, it shortens paths and increases edit distance, significantly reducing Accuracy. For example, consider the physical path $[n_1, n_2, n_3, n_5, n_6]$. If node n_3 remains silent, the virtual path becomes $[n_1, n_2, n_5, n_6]$. While Utility remains stable, Accuracy drops sharply, especially if n_3 was responsible for avoiding congestion or ensuring QoS.

2) *False response preserves Accuracy but harms Utility:* Replacing n_3 with a virtual node v_x results in $[n_1, n_2, v_x, n_5, n_6]$, which maintains structural length and thus only slightly decreases Accuracy. However, this disrupts prefix consistency, reducing the set of common links and lowering Utility. For instance, if a link failure occurs after n_5 , incorrect mapping prevents accurate detection, impairing event observability.

In summary, silence and false responses introduce a clear trade-off: silence favors Utility but sacrifices Accuracy, whereas false responses favor Accuracy at the expense of Utility. Optimizing α_n and β_n requires balancing these competing effects.

Building upon NetHide's two key observations, we introduce a new heuristic insight: a node's position within a path strongly influences its contribution to Utility. Replacing an early node drastically reduces prefix similarity and Utility, whereas replacing a later node has a smaller effect. For example, in $[n_1, n_2, n_3, n_4, n_5]$, replacing n_2 causes a significant Utility loss, while replacing n_4 has a minor impact.

To quantify this effect, we define a position-aware Utility score. For each physical path $R(f)$, if node $\mathcal{N}_i$ appears in the path, its contribution is updated as:

$$u_i = \sum_{f \in \mathcal{F}} \left(1 - \frac{\text{idx}(\mathcal{N}_i)}{|R(f)| - 1} \right), \quad (20)$$

where $\text{idx}(\mathcal{N}_i)$ is the index of $\mathcal{N}_i$ in $R(f)$ (starting from 0), and $|R(f)|$ is the path length. Intuitively, nodes closer to the start receive higher scores, as they are critical for maintaining failure observability. For Accuracy, we adopt the definition in

Algorithm 1. Computation of α_i and β_i for Node i .

Input: γ_i : probability of node i responding truthfully.
 $\mathcal{F}_i$: set of flows passing through node i .
Output: α_i, β_i : probabilities of the “silent” and “fake response” actions.

```

1 Step 1: Compute residual probability.
2  $r \leftarrow 1 - \gamma_i$ ;
3 if  $r \leq 0$  then
4   | return  $(\alpha_i \leftarrow 0, \beta_i \leftarrow 0)$ ;
5 end
6 Step 2: Initialize Utility and Accuracy scores.
7  $u_i \leftarrow 0, a_i \leftarrow 0$ ;
8 Step 3: Compute scores across flows.
9 foreach  $f \in \mathcal{F}_i$  do
10  | if  $\mathcal{N}_i \notin R(f)$  or  $|R(f)| \leq 2$  then
11  |   | continue;
12  | end
13  | # Compute Utility Score  $u_i$ .
14  |  $idx \leftarrow$  position of  $\mathcal{N}_i$  in  $R(f)$ ;
15  |  $u_i \leftarrow u_i + \left(1 - \frac{idx}{|R(f)|-1}\right)$ ;
16  | # Compute Accuracy Score  $a_i$ .
17  |  $R^v(f) \leftarrow R(f)$ ;
18  | Replace  $\mathcal{N}_i$  in  $R^v(f)$  with fake node  $n_x$ ;
19  |  $a_i \leftarrow a_i + acc(R(f), R^v(f))$ ;
20 end
21  $u_i \leftarrow u_i / |\mathcal{F}_i|, a_i \leftarrow a_i / |\mathcal{F}_i|$ ;
22 Step 4: Compute action scores and normalize.
23  $\epsilon_\alpha \leftarrow u_i \cdot (1 - a_i), \epsilon_\beta \leftarrow (1 - u_i) \cdot a_i$ ;
24 Apply softmax:  $[\alpha_{ratio}, \beta_{ratio}] \leftarrow \text{softmax}([\epsilon_\alpha, \epsilon_\beta])$ ;
25 Step 5: Scale by residual and output probabilities.
26  $\alpha_i \leftarrow r \cdot \alpha_{ratio}$ ;
27  $\beta_i \leftarrow r \cdot \beta_{ratio}$ ;
28 return  $(\alpha_i, \beta_i)$ ;

```

(7), treating each node’s contribution as its structural similarity weight.

Finally, we combine these contributions to compute α_n and β_n . For each node, we calculate:

- $\epsilon_\alpha = u_i \cdot (1 - a_i)$: favoring silence when Utility is critical but Accuracy impact is low.
- $\epsilon_\beta = (1 - u_i) \cdot a_i$: favoring false responses when Accuracy is critical but Utility impact is low.

Applying a softmax function over $[\epsilon_\alpha, \epsilon_\beta]$ yields normalized ratios, which are scaled by the residual probability $(1 - \gamma_n)$:

$$(\alpha_n, \beta_n) = (1 - \gamma_n) \cdot \text{softmax}(\epsilon_\alpha, \epsilon_\beta). \quad (21)$$

Intuitively, nodes that appear early in paths avoid false responses to preserve Utility, while nodes critical for routing fidelity avoid silence to maintain Accuracy. This adaptive allocation strikes a balance between the two usability metrics under the global exposure constraint. With this observation, each node can calculate the α_i and β_i values with the following algorithm 1.

Algorithm Explanation: Lines 1–2 compute the residual probability $r = 1 - \gamma_i$, which represents the remaining probability budget for obfuscation after accounting for truthful responses. If $r \leq 0$, node i always responds truthfully, so both α_i and β_i are

set to zero, and the algorithm terminates. Lines 3–5 initialize the accumulators u_i and a_i for Utility and Accuracy scores, respectively. These scores quantify the contribution of node i to the two usability metrics across all flows. Lines 6–14 iterate over the flows in $\mathcal{F}_i$. For each flow f , if node i does not appear in $R(f)$ or the path length is insufficient, the flow is skipped. Otherwise, the algorithm first computes the usability contribution of node i , assigning higher weights to positions closer to the path’s start using $1 - \frac{\text{idx}(\mathcal{N}_i)}{|R(f)|-1}$. Next, the Accuracy contribution is computed by replacing $\mathcal{N}_i$ with a fake node in $R^v(f)$ and evaluating the path similarity via $acc(R(f), R^v(f))$ based on Levenshtein distance. Line 15 normalizes u_i and a_i by $|\mathcal{F}_i|$ to obtain average scores. Lines 17–19 then compute two preference scores: $\epsilon_\alpha = u_i(1 - a_i)$, favoring silence when Utility is high but Accuracy is low, and $\epsilon_\beta = (1 - u_i)a_i$, favoring false replies when Accuracy is high but Utility is low. These scores are normalized using the softmax function to yield α_{ratio} and β_{ratio} . Finally, Lines 21–22 scale these ratios by the residual probability r , ensuring $\alpha_i + \beta_i = 1 - \gamma_i$, and output (α_i, β_i) .

This algorithm achieves a practical computational complexity of $O(|F_n|)$, where $|F_n|$ denotes the number of flows traversing a given node. In contrast, traditional approaches exhibit significantly higher complexity, such as $O(NF)$ [2], [3] or even $O(N!)$ [4]. By reducing complexity to a per-node scale, DynNet enables each node to compute its own policy efficiently, making the framework lightweight and well-suited for dynamic environments with frequent topology changes.

F. Hash-Based Mapping Implementation

The previous section described how each node in DynNet computes its obfuscation strategy as a probabilistic triplet $(\alpha_v, \beta_v, \gamma_v)$, representing the likelihood of remaining silent, replying deceptively, or responding truthfully when probed by topology-discovery flows. To efficiently apply these probabilities to active probing traffic, we propose a lightweight, hash-based mechanism for implementation at each node.

Step 1. Maintaining the Active Flow Table: Each node maintains a lightweight table of active flows, represented by a five-tuple: (source IP, destination IP, source port, destination port, protocol). This table tracks flow states and updates dynamically as flows change. The design imposes minimal overhead since similar structures exist in standard frameworks such as SDN flow tables [35], P4 match-action tables [40], and TCAM-based hardware. High-performance implementations can achieve $O(1)$ lookup complexity [70]. DynNet does not require a specific implementation but assumes efficient lookup and update operations.

Step 2. Hash-Based Action Mapping: For each probing flow f , the node computes a hash value:

$$H(f) = \text{Hash}(f.\text{five-tuple} \parallel \text{NodeInfo} \parallel \text{Salt}), \quad (22)$$

where $\text{Hash}(\cdot)$ can be any efficient hash function. Since the mechanism does not require cryptographic strength, lightweight options such as MurmurHash [71] are recommended over MD5 or SHA-1 for performance. Here, NodeInfo denotes a node-specific attribute (e.g., MAC address) to ensure node-level

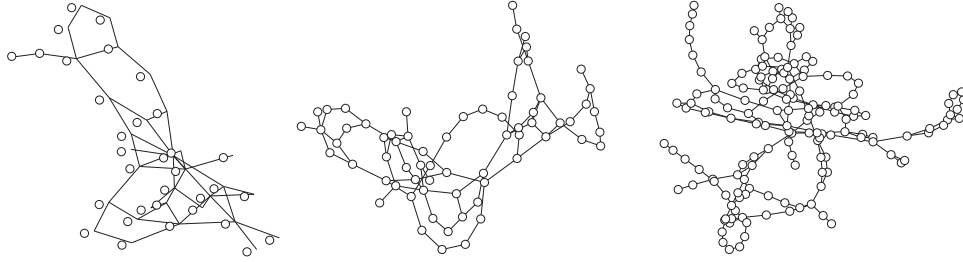


Fig. 3. Simulation topologies used in this study [72] (from left to right): Bics, a European backbone network with 33 nodes and 48 links. Missouri, a U.S. backbone network with 67 nodes and 83 links, and UsCarrier, a U.S. backbone network comprising 158 nodes and 189 links.

uniqueness, and `Salt` is a periodically refreshed random seed that introduces global re-randomization without affecting active sessions. We adopt an epoch-based design. Each new flow is hashed with the salt value of the current epoch and keeps using this value until the flow expires. This preserves consistent responses for repeated probes. The salt itself is refreshed on a slower timescale (*e.g.*, tens of minutes or hours), so the mapping changes over long attack windows while incurring negligible runtime overhead. The exact refresh interval is a configuration parameter: it should be longer than a typical probing session, yet shorter than the timescale over which an attacker can collect and exploit long-term statistics.

The hash space is then partitioned into three segments proportional to $(\alpha_v, \beta_v, \gamma_v)$:

$$\text{Range}_\alpha = [0, \alpha_v \cdot L), \quad (23)$$

$$\text{Range}_\beta = [\alpha_v \cdot L, (\alpha_v + \beta_v) \cdot L), \quad (24)$$

$$\text{Range}_\gamma = [(\alpha_v + \beta_v) \cdot L, L), \quad (25)$$

where L denotes the size of the hash space (*e.g.*, $L = 2^{32}$). The node determines its action by the range into which $H(f)$ falls:

- If $H(f) \in \text{Range}_\alpha$, the node remains silent by forwarding the TTL-expired packet without decrementing its TTL to conceal its presence. Consequently, the probing path effectively skips this node. For example, consider a probing path $[a, b, c, n, d]$, where node n receives a packet with $\text{TTL}=1$. According to the standard ICMP protocol [65], the node should decrement the TTL and send an ICMP `Time Exceeded` message to the source. Under DynNet's obfuscation policy, however, the node does not modify the TTL and forwards the packet to the next hop. As a result, the observed probing path becomes $[a, b, c, d]$, assuming other nodes respond truthfully.
- If $H(f) \in \text{Range}_\beta$, the node responds deceptively by sending an ICMP `Time Exceeded` message with a randomly selected fake IP address from a predefined pool. For instance, in the path $[a, b, c, n, d]$, if node n receives a packet with $\text{TTL}=1$, it decrements the TTL as usual but replies with a fabricated source IP, say x . The probing path then appears as $[a, b, x, c, d]$, assuming other nodes respond truthfully. The fake IP pool can be hardcoded into devices, and its design allows for different defense objectives. For example, SpiderNet [3] distributes fake IPs to create honeypot links for attack detection, while

EqualNet [2] assigns fake IPs within the same subnet as the node to improve utility. The design of the fake IP pool is highly deployment-specific and can follow existing best practices or customized policies. DynNet remains agnostic to this choice, ensuring compatibility with diverse strategies without constraining the operator's design space.

- If $H(f) \in \text{Range}_\gamma$, the node responds truthfully by following the ICMP standard and sending a valid ICMP `Time Exceeded` message with its real IP address.

This design offers two key benefits. First, it ensures consistency: for identical flow tuples, the hash-based mapping guarantees deterministic responses, preventing adversaries from exploiting inconsistencies. Second, it is lightweight and efficient: hash computation and range checking are computationally inexpensive and supported by commodity hardware and software platforms, making this approach suitable even for resource-constrained environments such as UAV networks. To further enhance security and flexibility, DynNet periodically refreshes the salt value, enabling controlled re-randomization of mappings without disrupting active sessions.

V. EVALUATION

This section presents an experimental evaluation of DynNet to assess its effectiveness in terms of both security and usability. All experiments were conducted on a server running Ubuntu 22.04, equipped with 200 GB RAM and a 98-core Intel(R) Xeon(R) Gold 6348 CPU @ 2.60 GHz.

We compare DynNet against two state-of-the-art NTO schemes: NetHide [1] and EqualNet [2]. As summarized in Fig. 3, these approaches share the same design objectives as DynNet, jointly addressing security and usability without differentiating between adversarial and benign probing traffic. Although other techniques exist in this space, their objectives differ significantly, making direct comparisons less meaningful. Since NetHide and EqualNet are not publicly available, we re-implemented both methods in Python based on their published descriptions and validated the correctness of our implementations through extensive testing. For a fair comparison of obfuscation strength, we do not match their control parameters directly. Instead, we normalize the effective obfuscation achieved by each scheme and discuss the mapping from DynNet's θ , NetHide's C , and EqualNet's τ in the corresponding evaluation subsections.

Our evaluation focuses on four aspects. First, we assess security by measuring the indistinguishability of nodes after

TABLE III
STD RESULTS FOR DIFFERENT METHODS UNDER VARYING θ

Topology	Method	$\theta = 0.1$	$\theta = 0.2$	$\theta = 0.3$	$\theta = 0.4$	$\theta = 0.5$	$\theta = 0.6$	$\theta = 0.7$	$\theta = 0.8$	$\theta = 0.9$
Bics	Gurobi [67]	2.55×10^{-9}	6.31×10^{-11}	1.87×10^{-11}	4.14×10^{-12}	5.78	16.74	30.53	47.25	68.86
	CVX [68]	4.07×10^{-15}	8.13×10^{-15}	1.39×10^{-14}	1.63×10^{-14}	5.78	16.74	30.53	47.25	68.86
	DynNet	3.09×10^{-16}	6.18×10^{-16}	1.24×10^{-15}	1.24×10^{-15}	4.18	10.59	16.96	23.45	29.84
Missouri	Gurobi [67]	1.06×10^{-10}	1.55×10^{-11}	4.78	30.48	62.05	98.96	145.45	207.27	285.49
	CVX [68]	2.21×10^{-14}	4.42×10^{-14}	4.78	30.48	62.05	98.96	145.45	207.27	285.49
	DynNet	1.49×10^{-14}	2.97×10^{-14}	4.01	22.73	42.87	63.84	83.74	103.67	121.20
UsCarrier	Gurobi [67]	2.72×10^{-11}	40.14	149.23	311.12	523.91	788.05	1102.37	1469.25	1892.69
	CVX [68]	1.75×10^{-13}	40.14	149.23	311.12	523.91	788.04	1102.36	1469.24	1892.67
	DynNet	1.01×10^{-14}	33.33	111.42	193.60	278.00	363.58	446.72	529.34	606.77

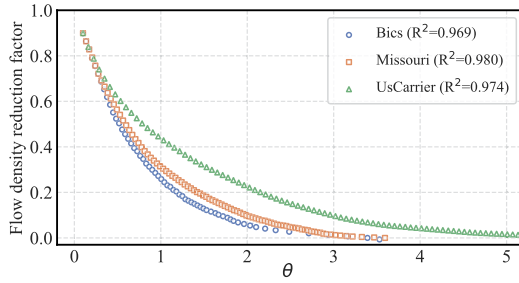


Fig. 4. Relationship between the obfuscation parameter θ and the flow density reduction factor.

obfuscation, expressed as the coefficient of variation of node exposure under equivalent flow-density reduction constraints. Second, we evaluate usability through Accuracy and Utility scores, which respectively capture path structural similarity and event observability. These metrics are widely adopted in prior NTO studies. Finally, we compare the computational overhead of DynNet against baseline schemes.

To ensure representative results, we rely on widely used benchmark topologies and simulation frameworks. Specifically, we select three real-world backbone topologies from the Internet Topology Zoo dataset [72]: Bics, Missouri, and UsCarrier. Their characteristics are summarized in Fig. 3. These topologies originate from North America and Europe and have been widely used in prior evaluations of topology obfuscation. To emulate dynamic environments, we simulate topology evolution by introducing random node movements while preserving node count and overall connectivity. During this process, links are added or removed at random to reflect realistic topology variations. Performance is evaluated periodically at multiple time steps, and final results are averaged across all iterations to ensure statistical robustness.

The remainder of this section details our experimental setup, evaluation metrics, and results.

A. Ablation Evaluation

As discussed in Section IV, the initial process in DynNet is to solve a convex optimization problem formulated in Equation (IV-D). In order to prevent unnecessary computational burdens, we came up with a practical approximation algorithm. To validate its efficacy, we compare our solution with two widely

used modern solvers: Gurobi [67] and CVX [68], [69], both of which are greatly optimized for convex problems and commonly applied in research and industry.

Table III outlines the STD of the node flow densities calculated by DynNet, Gurobi, and CVX for various obfuscation control parameter θ values. First, each of the three approaches achieves near-zero STD when θ is small (e.g., $\theta \leq 0.2$), demonstrating that dispersion minimization is straightforward under strong obfuscation. For example, for the Bics topology, DynNet achieves STDs of order 10^{-16} at $\theta = 0.1$, slightly better than CVX and Gurobi. This trend persists for all topologies with low θ , confirming that our heuristic achieves the same level of precision as state-of-the-art solvers in strong-obfuscation scenarios.

Second, as θ increases (i.e., less obfuscation), the performance difference between DynNet and convex solvers widens significantly in favor of DynNet. For instance, in the UsCarrier topology, both Gurobi and CVX report STDs above 1100 at $\theta = 0.7$, while DynNet reports 446.72. This gap increases at larger θ values. At $\theta = 0.9$, DynNet restricts STD to 606.77, whereas Gurobi has STD as high as 1892.69, proving DynNet's better capability in maintaining balanced flow distribution under loose constraints. In summary, these findings demonstrate that the practical approximation of DynNet outperforms conventional solvers at realistic parameter configurations and topologies at a large scale.

B. Parameter Analysis

To further understand the impact of the obfuscation parameter, Fig. 4 reports the relationship between θ and the flow density reduction factor on three topologies. In all cases, the reduction factor follows a smooth, monotonically decreasing and convex curve that is well approximated by a quadratic fit, with $R^2 > 0.96$. This indicates that small increases in θ yield almost linear improvements in flow-density reduction, whereas larger values of θ lead to diminishing returns and a gradual saturation. These results suggest that it is sufficient to operate DynNet in a moderate range of θ to obtain most of the security benefit without over-aggressively perturbing the topology.

C. Security Evaluation

As discussed in Section IV, the security objective is defined along two dimensions: (1) the *flow density reduction factor* and

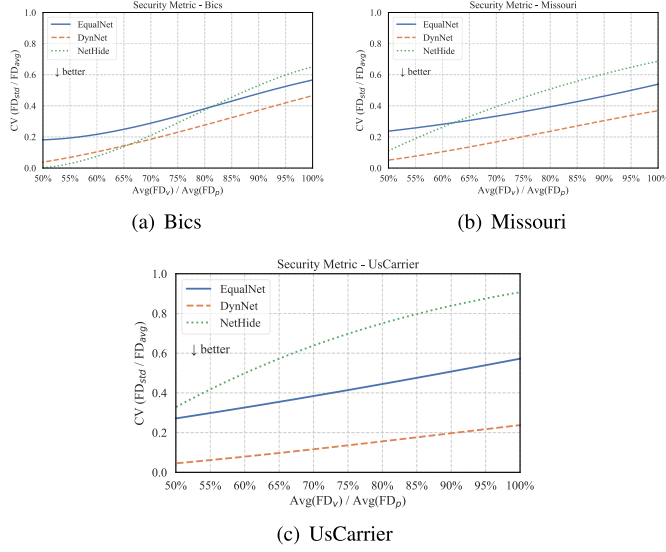


Fig. 5. Security performance comparison of DynNet, EqualNet [2], and NetHide [1] under varying obfuscation strengths across three real-world topologies.

(2) the normalized dispersion of node flow density, expressed as the coefficient of variation (CV).

The *flow density reduction factor* is not an absolute performance metric where higher or lower values inherently indicate superiority. Instead, it functions as a control parameter governing the degree of obfuscation, similar to the mechanisms adopted in other schemes. For example, NetHide regulates obfuscation through a constraint on the maximum number of flows per link (C), EqualNet employs a perturbation parameter τ , and DynNet uses an obfuscation control parameter θ . Because these knobs differ in scale and semantics, we normalize them through a common observable quantity: the ratio between the average virtual flow density and the average physical flow density. For each scheme, we sweep its control parameter over its valid range (DynNet's θ , NetHide's C , and EqualNet's $\tau \in [0, 1]$) and compute the resulting flow-density ratio. This ratio, ranging from 0.5 to 1.0 in our experiments, serves as the x -axis and provides a unified notion of obfuscation strength, where lower values indicate stronger obfuscation.

Fig. 5 compares DynNet with EqualNet and NetHide across three backbone topologies. Several observations emerge from these results. First, as the x -axis increases and the obfuscation level weakens, CV consistently rises for all three schemes. This trend aligns with expectations because lighter obfuscation prioritizes usability over security, which reduces the effectiveness of dispersion mitigation.

Second, the results highlight NetHide's sensitivity to network size. On small topologies such as Bics, NetHide achieves competitive performance and even outperforms DynNet in the 50%–65% obfuscation range, while surpassing EqualNet in most cases below 80%. However, as the topology size increases, this advantage rapidly diminishes. On medium-scale networks, NetHide only occasionally exceeds EqualNet, and on large-scale topologies, its performance degrades significantly compared to both DynNet and EqualNet. This decline stems from NetHide's

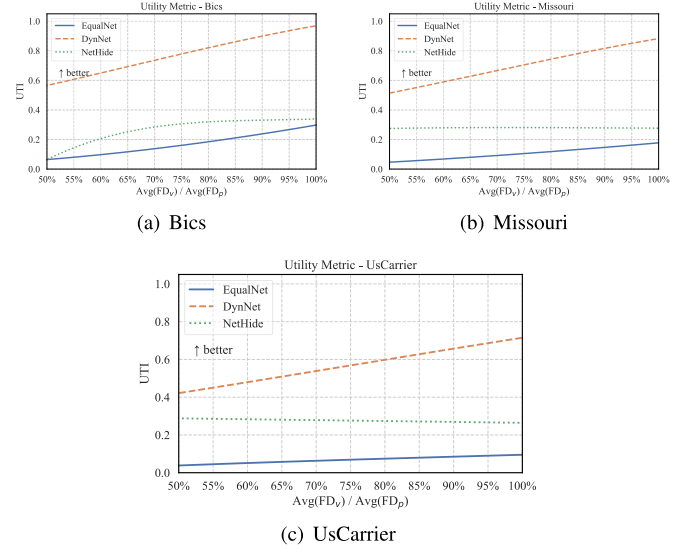


Fig. 6. Utility comparison of DynNet, EqualNet [2], and NetHide [1] under equivalent security conditions across three real-world topologies.

reliance on Monte-Carlo sampling, which generates numerous candidate virtual topologies and selects the best configuration. While effective for small graphs where exhaustive or near-exhaustive search is feasible, this approach incurs prohibitive computational costs as network size grows, limiting its scalability.

In contrast, DynNet demonstrates consistent and competitive security performance across all scenarios. It outperforms EqualNet under varying obfuscation levels and remains robust across different topology scales. Although NetHide occasionally surpasses DynNet in small-scale networks due to its exhaustive search capability, such cases represent isolated corner conditions rather than practical deployments. Overall, DynNet offers a favorable balance between security and scalability, outperforming EqualNet in all cases and delivering significantly greater practicality than NetHide.

D. Usability Evaluation

In this section, we evaluate the usability of DynNet. We begin by revisiting the usability objectives introduced in Section IV. Usability consists of two dimensions: Accuracy and Utility. Accuracy reflects the capability of legitimate users to perform routing over the virtual topology, while Utility captures the ability to observe critical events such as link failures within the obfuscated network. Both metrics are defined within the range $[0, 1]$, with higher values indicating stronger usability.

To ensure a fair comparison across schemes, we evaluate usability at matched obfuscation strengths. Concretely, we use the same flow-density ratio described in the security evaluation as the x -axis. This allows us to compare Accuracy under equivalent levels of security.

Fig. 6 illustrates the Utility performance of DynNet, EqualNet, and NetHide across three different topologies, represented by the orange dashed, blue solid, and green dotted lines, respectively. The results reveal several key trends. As the x -axis increases and obfuscation becomes weaker, EqualNet and

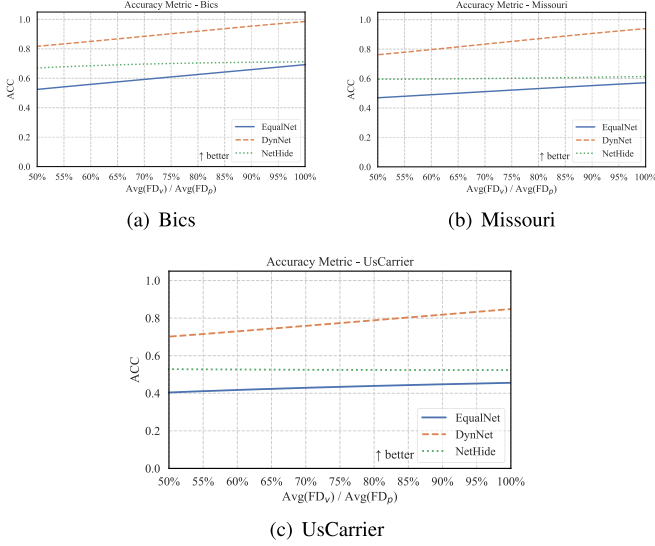


Fig. 7. Accuracy comparison of DynNet, EqualNet [2], and NetHide [1] under equivalent security conditions across three real-world topologies.

NetHide exhibit an upward trend in Utility, indicating improved event observability under lighter obfuscation. However, NetHide's performance remains inconsistent. While it achieves modest gains in small-scale topologies, its Utility stagnates in medium-scale networks and even declines in large-scale settings. This weakness stems from NetHide's reliance on a Monte Carlo-based approach, which struggles to adapt effectively under dynamic conditions and large-scale topologies. EqualNet, on the other hand, maintains a correct upward trend but consistently produces Utility scores significantly lower than those of NetHide and DynNet. In contrast, DynNet achieves superior and stable Utility across all scenarios, demonstrating its ability to maintain high observability without compromising security.

Fig. 7 presents the Accuracy results for the same set of topologies and security levels. The patterns largely mirror those observed for Utility. NetHide's performance deteriorates sharply as the topology size increases, reaffirming its lack of scalability. EqualNet shows a slightly better trend than in the Utility case, but still fails to deliver competitive Accuracy scores, reinforcing the usability trade-off embedded in its design. DynNet, by contrast, consistently outperforms both baselines, achieving higher Accuracy across all network sizes and obfuscation levels. This advantage is attributed to DynNet's lightweight and adaptive architecture, which efficiently updates obfuscation strategies as the network evolves. These findings collectively demonstrate that network topology obfuscation schemes do not necessarily require heavy computational complexity or exhaustive optimization to achieve a balanced trade-off between security and usability. DynNet exemplifies this principle by providing a scalable, practical, and high-performing solution for dynamic network environments.

E. Overhead Evaluation

We compare the running time of DynNet with two centralized NTO approaches: EqualNet and NetHide. We conduct experiments on three different network topologies with different sizes.

TABLE IV
RUNNING OVERHEAD (S) COMPARISON WITH BENCHMARKS

Topology Method	DynNet		EqualNet [2] (s)	NetHide [1] (s)
	Stage I (s)	Stage II (s)		
Bics	MAX 5.03×10^{-5}	1.73×10^{-2}	0.23	2.90
	MIN 3.46×10^{-5}	1.19×10^{-2}	0.08	2.01
	AVG 3.94×10^{-5}	1.26×10^{-2}	0.13	2.29
Missouri	MAX 6.49×10^{-5}	0.12	0.93	9.37
	MIN 3.72×10^{-5}	5.85×10^{-2}	0.42	7.55
	AVG 4.89×10^{-5}	7.16×10^{-2}	0.65	8.37
UsCarrier	MAX 1.58×10^{-4}	1.34	18.60	55.60
	MIN 7.01×10^{-5}	0.78	15.90	48.60
	AVG 1.04×10^{-4}	0.93	16.80	51.00

In addition to DynNet, running time is also separated into two parts: Stage I and Stage II. Because EqualNet and NetHide adopt centralized designs while DynNet is distributed, a direct runtime comparison could be unfair. To ensure fairness, all methods are executed in multi-core mode. For DynNet, the reported time corresponds to the worst-case runtime, defined as the elapsed time from the start of the first node to the completion of the last node. This metric provides an upper bound on practical deployment latency. Moreover, we configure all schemes to operate at comparable obfuscation strength: we set DynNet's $\theta = 0.5$ and EqualNet's $\tau = 0.5$, and choose NetHide's C such that the resulting flow-density ratio is close to 0.5, ensuring that the runtime comparison reflects similar security levels.

The overheads are summarized in Table IV. First, DynNet incurs negligible overhead in Stage I across all topologies, requiring less than 10^{-4} seconds even on the largest network and growing only marginally with network size. This confirms that security-oriented probability computation is computationally lightweight and highly scalable. Second, the most expensive computation cost of DynNet is through Stage II, which performs usability-driven reconfigurations for every flow passing through a particular node. As previously mentioned, Stage II has $O(|F_n|)$ complexity, where $|F_n|$ is the quantity of flows handled by node n . However, Stage II remains extremely efficient compared with benchmarks: its average run time is 0.0126 seconds for the small topology, 0.0716 seconds for the medium topology, and 0.925 seconds for the large topology. These results are far superior to centralized baselines. For example, on UsCarrier, DynNet runs both phases in approximately 0.93 seconds, whereas EqualNet and NetHide require 16.8 seconds and 51.0 seconds, respectively. Similar trends hold for smaller topologies, with DynNet achieving one to two orders of magnitude faster execution. Third, the scalability advantage of DynNet increases more sharply with the growth in network size. EqualNet's execution rises from 0.13 seconds on Bics to 16.8 seconds on UsCarrier, while NetHide rises from 2.29 seconds to over 51 seconds. In contrast, DynNet maintains overhead under one second even for the largest network, underscoring its suitability for dynamic environments requiring frequent reconfiguration.

VI. DISCUSSION

Beyond ICMP-based TTL probing, a more powerful adversary may attempt to infer routing and critical links via RTT

tomography. These methods do not rely on ICMP responsiveness. Instead, they exploit the repeatability of end-to-end delay structure and stable path-level correlations across measurements to attribute observed RTT traces to underlying links. DynNet's per-node probabilistic control already disrupts the repeatability of traffic-to-path realizations, which weakens correlation-based inference by reducing measurement consistency across snapshots. As a future direction, DynNet can be further strengthened by augmenting each participating node with a lightweight, fully distributed delay-perturbation module against such attacks. Operationally, each node would maintain a short virtual queue per outgoing interface. This introduces additional delay and jitter to disrupt correlation analysis. Since this perturbation is local and distributed, and it does not require controller-wide coordination, it fits within DynNet's design framework and can be integrated without architectural changes. However, optimizing the timing and magnitude of the injected perturbation requires careful analysis. The design must preserve QoS constraints, avoid amplifying congestion, and remain robust under diverse traffic regimes. Future work may necessitate a new optimization formulation to trade off security gains against delay inflation.

VII. CONCLUSION

In this paper, we presented DynNet, a lightweight and distributed NTO framework designed for dynamic and decentralized network environments. Unlike prior centralized or high-overhead approaches, DynNet enables each node to independently respond to probing packets using precomputed probabilistic strategies, eliminating the need for global coordination or real-time optimization. We solve the obfuscation process as a two-stage problem. The first stage balances exposure across nodes by optimizing each node's truthful response probability under time-varying topologies. The second stage decomposes this probability into concrete actions, deception, silence, or truth. To efficiently enforce these strategies at runtime, we introduced a hash-based dispatch mechanism that maps flows to obfuscation actions deterministically and consistently, while preserving the target distribution. Through comprehensive simulations on real-world topologies from the Topology Zoo dataset, we demonstrated that DynNet achieves competitive or superior performance compared to state-of-the-art NTO baselines in terms of security and usability, while reducing runtime overhead.

REFERENCES

- [1] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and practical network topology obfuscation," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 693–709.
- [2] J. Kim, E. Marin, M. Conti, and S. Shin, "EqualNet: A secure and practical defense for long-term network topology obfuscation," in *Proc. 29th Netw. Distrib. Syst. Secur. Symp.*, 2022, pp. 1–18.
- [3] X. Huang et al., "SpiderNet: Enabling bot identification in network topology obfuscation against link flooding attacks," *IEEE Trans. Netw.*, vol. 33, no. 1, pp. 99–113, Feb. 2025.
- [4] "CISO panel: The future of cyber is automated moving target defense," 2023. Accessed: Feb. 2026. [Online]. Available: <https://www.youtube.com/watch?v=PxxY4IZXSUE>
- [5] C. C. Manion, "The emerging moving target defense market—what you need to know," 2023. Accessed: Feb. 2026. [Online]. Available: <https://www.linkedin.com/pulse/emerging-moving-target-defense-market-what-you-need-carl-c-manion>
- [6] C. M. Mark Pohto, "Gartner emerging tech: Security—Tech innovators in automated moving target defense," 2023. Accessed: 2025. [Online]. Available: <https://www.gartner.com/en/documents/6684534>
- [7] M. S. Kang, S. B. Lee, and V. D. Gligor, "The crossfire attack," in *Proc. 34th IEEE Symp. Secur. Privacy*, 2013, pp. 127–141.
- [8] X. Huang et al., "You can obfuscate, but you cannot hide: CrossPoint attacks against network topology obfuscation," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 5735–5750.
- [9] "Traceroute - Print the route packets trace to network host," 2006. Accessed: Feb. 2026. [Online]. Available: <https://linux.die.net/man/8/traceroute>
- [10] "Nmap: The network mapper - free security scanner," 1998. Accessed: Feb. 2026. [Online]. Available: <https://nmap.org/>
- [11] X. Ding, F. Xiao, M. Zhou, and Z. Wang, "Active link obfuscation to thwart link-flooding attacks for Internet of Things," in *Proc. 19th IEEE Int. Conf. Trust Secur. Privacy Comput. Commun.*, 2020, pp. 217–224.
- [12] T. Hou, Z. Qu, T. Wang, Z. Lu, and Y. Liu, "ProTO: Proactive topology obfuscation against adversarial network topology inference," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun.*, 2020, pp. 1598–1607.
- [13] J. Kim, J. Nam, S. Lee, V. Yegneswaran, P. Porras, and S. Shin, "BottleNet: Hiding network bottlenecks using SDN-based topology deception," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 3138–3153, 2021.
- [14] Y. Liu, J. Zhao, G. Zhang, and C. Xing, "NetObfu: A lightweight and efficient network topology obfuscation defense scheme," *Comput. Secur.*, vol. 110, 2021, Art. no. 102447.
- [15] Z. Zhu, G. Zhu, Y. Zhang, J. Shi, X. Huang, and Y. Fang, "EigenObfu: A novel network topology obfuscation defense method," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 1, pp. 451–462, Jan./Feb. 2025.
- [16] "The DDoS that almost broke the internet," 2013. Accessed: Feb. 2026. [Online]. Available: <https://blog.cloudflare.com/the-ddos-that-almost-broke-the-internet/>
- [17] A. Studer and A. Perrig, "The Coremelt attack," in *Proc. 14th Eur. Symp. Res. Comput. Secur.*, 2009, pp. 37–52.
- [18] J. Wang, R. Wen, J. Li, F. Yan, B. Zhao, and F. Yu, "Detecting and mitigating target link-flooding attacks using SDN," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 6, pp. 944–956, Nov./Dec. 2019.
- [19] C. Liaskos, V. Kotronis, and X. Dimitropoulos, "A novel framework for modeling and mitigating distributed link flooding attacks," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun.*, 2016, pp. 1–9.
- [20] Z. Huang et al., "Defending against link-flooding attacks with adversary interest prediction and grouped online load balancing," *IEEE Trans. Netw.*, vol. 33, no. 3, pp. 1056–1069, Jun. 2025.
- [21] S. B. Lee, M. S. Kang, and V. D. Gligor, "CoDef: Collaborative defense against large-scale link-flooding attacks," in *Proc. 9th ACM Conf. Emerg. Netw. Experiments Technol.*, 2013, pp. 417–428.
- [22] M. S. Kang et al., "SPIFFY: Inducing cost-detectability tradeoffs for persistent link-flooding attacks," in *Proc. 23rd Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 53–55.
- [23] M. Tran, M. S. Kang, H.-C. Hsiao, W.-H. Chiang, S.-P. Tung, and Y.-S. Wang, "On the feasibility of rerouting-based DDoS defenses," in *Proc. 40th IEEE Symp. Secur. Privacy*, 2019, pp. 1169–1184.
- [24] N. G. Duffield and F. L. Presti, "Network tomography from measured end-to-end delay covariance," *IEEE/ACM Trans. Netw.*, vol. 12, no. 6, pp. 978–992, Dec. 2004.
- [25] M. Mardani and G. B. Giannakis, "Estimating traffic and anomaly maps via network tomography," *IEEE/ACM Trans. Netw.*, vol. 24, no. 3, pp. 1533–1547, Jun. 2016.
- [26] I. Sartzetakis and E. Varvarigos, "Machine learning network tomography with partial topology knowledge and dynamic routing," in *Proc. 65th IEEE Glob. Commun. Conf.*, 2022, pp. 4922–4927.
- [27] I. Sartzetakis and E. Varvarigos, "Network tomography with partial topology knowledge and dynamic routing," *J. Netw. Syst. Manage.*, vol. 31, no. 4, 2023, Art. no. 73.
- [28] A. Rkhami, Y. Hadjadj-Aoul, G. Rubino, and A. Outagarts, "On the use of machine learning and network tomography for network slices monitoring," in *Proc. 22nd Int. Conf. High Perform. Switching Routing*, 2021, pp. 1–7.
- [29] C. Du, Z. Yu, and X. Wang, "Identification of path congestion status for network performance tomography using deep spatial-temporal learning," *Comput. Commun.*, vol. 239, 2025, Art. no. 108194.
- [30] H. Ikeuchi and H. Saito, "Network tomography based on adaptive measurements in probabilistic environments," *IEEE Trans. Inf. Theory*, vol. 70, no. 11, pp. 8251–8265, Nov. 2024.

- [31] X. Xu, X. Zhu, and C. Zhu, "GAN-based deep learning framework of network reconstruction," *Complex Intell. Syst.*, vol. 9, no. 3, pp. 3131–3146, 2023.
- [32] C. Liaskos and S. Ioannidis, "Network topology effects on the detectability of crossfire attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 7, pp. 1682–1695, Jul. 2018.
- [33] X. Ma et al., "Randomized security patrolling for link flooding attack detection," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 4, pp. 795–812, Jul./Aug. 2020.
- [34] Y. Li et al., "A collaborative programmable LFA defense using temporal graph learning in AIoT," *IEEE Internet Things J.*, vol. 12, no. 3, pp. 2514–2529, Feb. 2025.
- [35] N. McKeown et al., "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008.
- [36] L. Xue, X. Ma, X. Luo, E. W. W. Chan, T. T. N. Miu, and G. Gu, "LinkScope: Toward detecting target link flooding attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 10, pp. 2423–2438, Oct. 2018.
- [37] N. Ravi, S. M. Shalinie, and D. D. Jose Theres, "BALANCE: Link flooding attack detection and mitigation via hybrid-SDN," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 3, pp. 1715–1729, Sep. 2020.
- [38] J. Zheng, Q. Li, G. Gu, J. Cao, D. K. Y. Yau, and J. Wu, "Realtime DDoS defense using COTS SDN switches via adaptive correlation analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 7, pp. 1838–1853, Jul. 2018.
- [39] X. Jiang et al., "Credible link flooding attack detection and mitigation: A blockchain-based approach," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3537–3554, Jun. 2024.
- [40] P. Bosshart et al., "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 87–95, 2014.
- [41] Y. Tan, J. Liu, and J. Wang, "How to protect key drones in unmanned aerial vehicle networks? An SDN-based topology deception scheme," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13320–13331, Dec. 2022.
- [42] C. Qiu, B. Ren, G. Tang, L. Luo, and D. Guo, "ChameleonNet: Topology obfuscation against tomography with critical information hiding," *IEEE Trans. Netw.*, vol. 33, no. 5, pp. 2585–2600, Oct. 2025.
- [43] T. Hou, T. Wang, Z. Lu, and Y. Liu, "Combating adversarial network topology inference by proactive topology obfuscation," *IEEE/ACM Trans. Netw.*, vol. 29, no. 6, pp. 2779–2792, Dec. 2021.
- [44] L. Wang, Q. Li, Y. Jiang, X. Jia, and J. Wu, "Woodpecker: Detecting and mitigating link-flooding attacks via SDN," *Comput. Netw.*, vol. 147, pp. 1–13, 2018.
- [45] A. Rezapour and W.-G. Tzeng, "RL-Shield: Mitigating target link-flooding attacks using SDN and deep reinforcement learning routing algorithm," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 4052–4067, Nov./Dec. 2022.
- [46] M. Zhang et al., "Poseidon: Mitigating volumetric DDoS attacks with programmable switches," in *Proc. 27th Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.
- [47] Z. Liu et al., "Jaquen: A high-performance switch-native approach for detecting and mitigating volumetric DDoS attacks with programmable switches," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 3829–3846.
- [48] J. Xing, W. Wu, and A. Chen, "Ripple: A programmable, decentralized link-flooding defense against adaptive adversaries," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 3865–3881.
- [49] H. Zhou, S. Hong, Y. Liu, X. Luo, W. Li, and G. Gu, "Mew: Enabling large-scale and dynamic link-flooding defenses on programmable switches," in *Proc. 43rd IEEE Symp. Secur. Privacy*, 2022, pp. 1625–1639.
- [50] M. Nance-Hall, Z. Liu, V. Sekar, and R. Durairajan, "Analyzing the benefits of optical topology programming for mitigating link-flood DDoS attacks," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 146–163, Jan./Feb. 2025.
- [51] C. Basescu et al., "SIBRA: Scalable internet bandwidth reservation architecture," in *Proc. 23rd Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–16.
- [52] J. M. Smith and M. Schuchard, "Routing around congestion: Defeating DDoS attacks and adverse network conditions via reactive BGP routing," in *Proc. 39th IEEE Symp. Secur. Privacy*, 2018, pp. 599–617.
- [53] N. Lu, J. Zhang, X. Liu, W. Shi, and J. Ma, "STOP: A service oriented internet purification against link flooding attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 938–953, 2022.
- [54] P. Beltrán-López, M. G. Pérez, and P. Nespole, "Cyber deception: Taxonomy, state of the art, frameworks, trends, and open challenges," *IEEE Commun. Surveys Tuts.*, vol. 28, pp. 1520–1556, 2026.
- [55] Y. Liu, C. Xing, G. Zhang, L. Song, and H. Lin, "AntiTomo: Network topology obfuscation against adversarial tomography-based topology inference," *Comput. Secur.*, vol. 113, 2022, Art. no. 102570.
- [56] X. Li, X. Yang, Y. Huang, and Y. Chen, "Combating temporal composition inference by high-order camouflaged network topology obfuscation," *Comput. Secur.*, vol. 144, 2024, Art. no. 103981.
- [57] X. Yang, Z. Zhong, Y. Chen, X. Li, J. Wang, and Z. Cai, "Toward dynamic topology obfuscation for IoT networks with evolutionary defense," *IEEE Internet Things J.*, vol. 12, no. 21, pp. 45848–45866, Nov. 2025.
- [58] S. Achleitner, T. F. La Porta, P. McDaniel, S. Sugrim, S. V. Krishnamurthy, and R. Chadha, "Deceiving network reconnaissance using SDN-based virtual topologies," *IEEE Trans. Netw. Service Manag.*, vol. 14, no. 4, pp. 1098–1112, Dec. 2017.
- [59] X. Qin, F. Jiang, X. Qin, L. Ge, M. Lu, and R. Doss, "CGAN-based cyber deception framework against reconnaissance attacks in ICS," *Comput. Netw.*, vol. 251, 2024, Art. no. 110655.
- [60] H. Alanazi, S. Bi, T. Wang, and T. Hou, "Adaptive feature engineering via attention-based LSTM towards high performance reconnaissance attack detection," in *Proc. IEEE Mil. Commun. Conf.*, 2023, pp. 542–547.
- [61] Y. Wang, Z. Su, A. Benslimane, Q. Xu, M. Dai, and R. Li, "Collaborative honeypot defense in UAV networks: A learning-based game approach," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1963–1978, 2024.
- [62] M. Dai, Z. Su, Q. Xu, Y. Wang, and N. Lu, "A trust-driven contract incentive scheme for mobile crowd-sensing networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 2, pp. 1794–1806, Feb. 2022.
- [63] Y. Wang, Z. Su, N. Zhang, and A. Benslimane, "Learning in the air: Secure federated learning for UAV-assisted crowdsensing," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1055–1069, Apr.–Jun. 2021.
- [64] mtr(8)–linux manual page, 1997. Accessed: Feb. 2026. [Online]. Available: <https://man7.org/linux/man-pages/man8/mtr.8.html>
- [65] J. Postel, "Internet control message protocol," *RFC 792*, 1981. Accessed: Feb. 2026. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc792>
- [66] V. Lcvenshtcin, "Binary coors capable of correcting deletions, insertions, and reversals," *Sov. Phys.-Doklady*, vol. 10, pp. 707–710, 1966.
- [67] Gurobi Optimization, LLC, "Gurobi optimizer reference manual," 2024. Accessed: Feb. 2026. [Online]. Available: <https://www.gurobi.com>
- [68] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," 2012. Accessed: Feb. 2026. [Online]. Available: <https://cvxr.com/cvx>
- [69] M. Grant and S. Boyd, "Graph implementations for nonsmooth convex programs," in *Proc. Recent Adv. Learn. Control*, 2008, pp. 95–110. [Online]. Available: http://stanford.edu/boyd/graph_dcp.html
- [70] Y. Wan et al., "FastUp: Fast TCAM update for SDN switches in datacenter networks," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 887–897.
- [71] A. Appleby, "Murmurhash3," 2011. Accessed: Feb. 2026. [Online]. Available: <https://github.com/aappleby/smhasher>
- [72] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The internet topology zoo," *IEEE J. Sel. Areas Commun.*, vol. 29, no. 9, pp. 1765–1775, Oct. 2011.



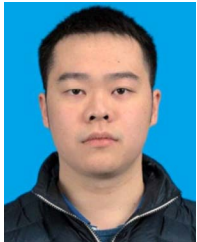
Xuanbo Huang (Member, IEEE) received the bachelor's and Ph.D. degrees in the major of information security from the School of Cyber Science and Technology, University of Science and Technology of China, Hefei, China, in 2020 and 2025, respectively. His research interests include security issues in software-defined networking and distributed denial of service attacks.



Kaiping Xue (Senior Member, IEEE) received the bachelor's degree from the Department of Information Security, University of Science and Technology of China (USTC), Hefei, China, in 2003, and the Ph.D. degree from the Department of Electronic Engineering and Information Science, USTC, in 2007. From 2012 to 2013, he was a Postdoctoral Researcher with the Department of Electrical and Computer Engineering, University of Florida. He is currently a Professor with the School of Cyber Science and Technology, USTC. He is also the Director of Network and Information Center, USTC. His research interests include next-generation Internet architecture design, transmission optimization, and network security. His work won best paper awards in IEEE MSN 2017 and IEEE HotICN 2019, IEEE ICC 2025, Best Paper Honorable Mention in ACM CCS 2022, Best Paper Runner-Up Award in IEEE MASS 2018, and best track paper in MSN 2020. He serves on the Editorial Board of several journals, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, and IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT. He has served as the Guest Editor of many reputed journals/magazines, and Track/Symposium Chair for some reputed conferences. He is an IET Fellow.



Lutong Chen (Member, IEEE) received the bachelor's and Ph.D. degrees in the major of information security from the School of Cyber Science and Technology, University of Science and Technology of China, Hefei, China, in 2020 and 2024, respectively. He is currently an Engineer with Network and Information Center, University of Science and Technology of China. His research interests include network security and quantum networks.



Zixu Huang received the bachelor's degree in computer science and technology in 2021 from the School of the Gifted Young, University of Science and Technology of China (USTC), Hefei, China, where he is currently working toward the Ph.D. degree in the major of information security with the School of Cyber Science and Technology. His research interests include addressing network threats using software-defined networking and programmable switch.



Jian Li (Senior Member, IEEE) received the bachelor's degree from the Department of Electronics and Information Engineering, Anhui University, Hefei, China, in 2015, and the Ph.D. degree from the Department of Electronic Engineering and Information Science, University of Science and Technology of China (USTC), Hefei, in 2020. From 2019 to 2020, he was a Visiting Scholar with the Department of Electronic and Computer Engineering, University of Florida. From 2020 to 2022, he was a Postdoctoral Researcher with the School of Cyber Science and Technology, USTC. He is currently an Associate Researcher with the School of Cyber Science and Technology, USTC. His research interests include quantum networking, wireless networks, and network security. He serves as the Editor of IEEE TRANSACTIONS ON COMMUNICATIONS and CHINA COMMUNICATIONS, and Technical Program Committee Co-Chair for IEEE IWCNC 2025 QNC Symposium. He was the recipient of IEEE ComSoc Internet Technical Committee Early Career Award 2025.



Hyundong Shin (Fellow, IEEE) received the B.S. degree in electronics engineering from Kyung Hee University (KHU), Yongin-si, South Korea, in 1999, and the M.S. and Ph.D. degrees in electrical engineering from Seoul National University, Seoul, South Korea, in 2001 and 2004, respectively. During his Postdoctoral Research with the Massachusetts Institute of Technology from 2004 to 2006, he was with the Laboratory for Information Decision Systems. In 2006, he joined KHU, where he is currently a Professor with the Department of Electronic Engineering. His research interests include quantum information science, wireless communication, and machine intelligence. Dr. Shin was the recipient of the IEEE Communications Society's Guglielmo Marconi Prize Paper Award and William R. Bennett Prize Paper Award. He served as the Publicity Co-Chair for the IEEE PIMRC and Technical Program Co-Chair for the IEEE WCNC and the IEEE GLOBECOM. He was the Editor of IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS and IEEE COMMUNICATIONS LETTERS.